

SZABLON RAPORTU PROJEKTOWEGO

Egzamin eksternistyczny – Paradygmaty Programowania

Projekt grupowy (3 - 4 osoby) | Python / Java / C++

Tytuł projektu:	AeroSimulator
Numer grupy:	8
Imię i nazwisko – Student 1:	Tomasz Orman
Imię i nazwisko – Student 2:	Michał Iwanejko
Imię i nazwisko – Student 3:	Jakub Górski
Imię i nazwisko – Student 4:	Przemysław Jaworski
Język/i programowania:	C#
Zastosowany/e paradygmat/y:	hybryda - OOP + funkcyjny
Data złożenia raportu:	22.06.2026

0. Wstęp i instrukcja dla grupy projektowej

Dokument ten stanowi obowiązujący szablon raportu końcowego dla grupowego projektu zaliczeniowego z przedmiotu Paradygmaty Programowania. Każda grupa zobowiązana jest wypełnić wszystkie sekcje zgodnie z poniższymi wskazówkami, zachowując tę strukturę i formatowanie. Raport musi być złożony w formacie .docx lub .pdf.

0.1. Cel i zakres projektu

Projekt ma na celu wykazanie, że studenci potrafią zastosować wiedzę zdobytą na zajęciach w praktycznym, nietrywialnym problemie. Projekt powinien być funkcjonalnie rozbudowany, a wybrany paradygmat (lub ich kombinacja) musi być uzasadniony charakterem problemu, a nie wyłącznie znajomością języka.

0.2. Ogólne wymagania formalne

- **Rozmiar grupy:** 3 – 4 osoby (każda z przypisanym jednoznacznym wkładem).
- **Języki programowania:** Python, Java, C++ lub inny (możliwe połączenie).
- **Paradygmaty:** OOP, funkcyjny, programowanie wielowątkowe lub **hybryda OOP + funkcyjny (preferowane)**.
- **Testy:** mile widziane –jednostkowe (unit tests) lub inne.
- **Format raportu:** .docx lub .pdf
- **Termin:** zgodnie z harmonogramem ogłoszonym przez prowadzącego.

WAŻNE

Projekty, które nie spełniają kryterium uzasadnienia paradygmatu, lub kryterium udokumentowania indywidualnego wkładu – nie będą przyjęte do oceny.

1. Opis projektu

1.1. Tytuł i ogólny opis

Podaj pełny tytuł projektu oraz zwięzły opis tego, czym projekt się zajmuje, jaki problem rozwiązuje i dlaczego jest to zagadnienie interesujące z inżynierskiego lub naukowego punktu widzenia.

Tytuł projektu: AeroSimulator - konsolowy symulator lotu i awarii kaskadowych samolotu.

Opis projektu: AeroSimulator jest aplikacją konsolową napisaną w języku C#, która symuluje lot samolotu w czasie rzeczywistym. Użytkownik wybiera model samolotu, trasę i poziom trudności, a następnie steruje parametrami lotu takimi jak przepustnica, pochylenie, kurs, autopilot, podwozie, klapy oraz systemy pokładowe. Projekt nie ogranicza się do prostego przemieszczania obiektu po ekranie, modeluje także zależności między systemami samolotu czujnikami, paliwem, silnikami, hydrauliką, pogodą i losowymi anomaliami. Najważniejszą cechą projektu jest system awarii kaskadowych, w którym pojedyncze zdarzenie może powodować dalsze skutki, np. bird strike może doprowadzić do pożaru silnika, pożar do uszkodzenia skrzydła, a to do asymetrycznego znoszenia i utraty kontroli.

Z inżynierskiego punktu widzenia projekt jest interesujący, ponieważ łączy symulację dynamicznego systemu, architekturę zdarzeniową, wzorce projektowe oraz modelowanie stanów maszyny. To zagadnienie interesujące naukowo, ponieważ wymaga bezbłędnego połączenia paradygmatu obiektowego (do enkapsulacji stanu komponentów) z paradygmatem funkcyjnym (do bezpiecznego, niemutowalnego przesyłania danych).

1.2. Motywacja i kontekst problemu

Wyjaśnij, dlaczego wybraliście ten konkretny problem. Opisz jego praktyczne znaczenie, potencjalnych użytkowników lub dziedzinę zastosowania (np. analiza danych, bioinformatyka, finanse, gry, automatyka, bezpieczeństwo IT, przetwarzanie języka naturalnego itp.). Opisz stan istniejących rozwiązań (jeśli dotyczy) i wskaż, co wasza implementacja wnosi nowego.

Wybraliśmy problem symulacji lotniczej, ponieważ naturalnie wymusza ona rozdzielanie odpowiedzialności na wiele współpracujących, ale niezależnie testowalnych podsystemów — model fizyki, awionikę, czujniki, sterowanie, logikę zdarzeń losowych oraz interfejs tekstowy — co jest dokładnie tym, co projekt miał demonstrować w praktyce, a nie tylko w teorii. Domena lotnicza jest też na tyle bogata regułowo (procedury awaryjne, ograniczenia prędkości przeciągnięcia i przekroczenia, fazy lotu, listy kontrolne), że dawała wystarczająco dużo możliwości do zaprojektowania realnej hierarchii stanów i strategii, bez konieczności sztucznego rozbudowywania zakresu.

Praktyczne znaczenie projektu jest edukacyjne (wizualizacja własnych decyzji pilota i ich konsekwencji w czytelnym, tekstowym kokpicie) oraz inżynierskie jako poligon do eksperymentowania z architekturą systemów reaktywnych czasu rzeczywistego. Wzorce zastosowane tutaj (Stan, Strategia, Obserwator, Komenda, Fabryka) są tymi samymi wzorcami, które są używane w grach symulacyjnych czy oprogramowaniu wbudowanym sterującym urządzeniami o wielu trybach pracy.

Nie bazowaliśmy na istniejącym, konkretnym projekcie referencyjnym, lecz zaprojektowaliśmy własny, uproszczony model fizyki lotu i własny katalog 13 typów anomalii oraz 6 strategii pogodowych. Nowością wniesioną przez implementację jest połączenie klasycznego, w pełni obiektowego modelu maszyny stanów z funkcyjną warstwą raportowania zdarzeń (deklaratywna agregacja LINQ nad niemutowalnym dziennikiem zdarzeń) oraz autorska, minimalistyczna monada `Option<T>` użyta konsekwentnie w całej warstwie czujników, zamiast tradycyjnego zwracania wartości sentinel (np. -1) lub zgłaszania wyjątków przy martwym czujniku.

1.3. Cele funkcjonalne systemu

Wymień co najmniej 5 konkretnych, mierzalnych celów funkcjonalnych projektu.

1. System inicjalizuje konfigurowalny model samolotu na podstawie wybranego presetu oraz trasy (**AircraftFactory**).
2. System prowadzi pętlę symulacji w czasie rzeczywistym z krokiem dt (domyślnie 0,1 s), aktualizując fizykę, systemy, pogodę i widoki, bez akumulacji opóźnień.

3. System zarządza fazami lotu poprzez klasyczną maszynę stanów (10 klas implementujących **IAircraftState**, m.in. TakeOff, Cruise, Emergency, Landing).
4. System asynchronicznie komunikuje podsystemy poprzez centralną szynę zdarzeń (**EventBus**), realizując bezpieczne dla typów przekazywanie informacji (np. logowanie zdarzeń do "czarnej skrzynki").
5. System generuje w czasie rzeczywistym i obsługuje kaskadowe propagowanie awarii (np. wyciek hydrauliki → zacięcie podwozia) w oparciu o obiekt **CascadeHandler**.
6. System wymusza obsługę niedokładności pomiarowej – każdy z czujników nakłada szum proporcjonalny do trudności i stanu zdrowia sprzętu, zwracając wynik w opakowaniu **Option<double>**.
7. System przelicza parametry korekcyjne autopilota przy użyciu bezpiecznych, czystych funkcji matematycznych bez bezpośrednich efektów ubocznych (**AutopilotCalculator**).
8. System oferuje mechanizm odwoływania akcji pilota (Undo/Redo) dla krytycznych systemów przy użyciu wzorca Command (**CommandHistory**).
9. System ocenia jakość lądowania na podstawie parametrów fizycznych (prędkość opadania, przechylenie) i jednoznacznie klasyfikuje wynik, generując statystyczny raport końcowy (potoki LINQ nad dziennikiem zdarzeń).

1.4. Wymagania niefunkcjonalne

Opisz wymagania dotyczące jakości systemu, takie jak: wydajność (np. w przypadku wielowątkowego), skalowalność, modularność, testowalność, czytelność kodu i zgodność z zasadami SOLID (jeśli dotyczy).

- **Wydajność:** Pętla symulacji mieści się w rygorystycznym budżecie czasowym. Narzut obliczeniowy generowany przez wzorzec zdarzeniowy (**EventBus**) i logikę kaskadową został zoptymalizowany. Projekt minimalizuje wielowątkowość dla obliczeń fizycznych, rezerwując operacje async/await do utrzymania responsywności wejścia i generowania płynnego interfejsu tekstowego.
- **Modularność i rozszerzalność: (Zgodność z Open/Closed):** Dodanie nowych anomalii (implementacja **IAnomaly**), systemów pogodowych (**IWeatherStrategy**) czy komend sterujących (**IFlightCommand**) nie wymaga modyfikowania istniejącego kodu kontrolerów symulacji.
- **Testowalność:** Logika domenowa (katalog Core) jest ściśle odseparowany od interfejsu (katalog Views). Logikę wyodrębniono do czystych funkcji, co pozwoliło pokryć projekt zestawem 25 testów jednostkowych i integracyjnych (xUnit + FluentAssertions + coverlet).
- **Czytelność i dokumentacja kodu:** Konsekwentne nazewnictwo (sufiksy ról wzorcowych), rygorystyczny tryb Nullable reference types oraz użycie niemutowalnych struktur (rekordy takie jak FlightDataSnapshot do eksportu raportów). Eliminuje to ryzyko błędów współbieżności przy odczycie telemetrycznym. Najważniejsze moduły są opisane poprzez komentarze w kodzie.

- **Niezawodność i bezpieczeństwo stanu:** Zrealizowana architektura hybrydowa minimalizuje ryzyko wystąpienia błędów wynikających z nieoczekiwanej mutacji współdzielonego stanu (tzw. wyścigi danych). Wykorzystanie paradygmatu funkcyjnego w komunikacji asynchronicznej sprawiło, że zachowanie systemu w obliczu wysoce krytycznych, kaskadowych awarii jest stabilne i przewidywalne.

2. Uzasadnienie wyboru paradygmatu programowania

Jest to jedna z kluczowych sekcji raportu. Musicie wykazać, że wybrany paradygmat (lub ich kombinacja) wynika z natury problemu, a nie jest przypadkowy. Niepełne uzasadnienie skutkuje obniżeniem oceny.

2.1. Wybrany paradygmat / kombinacja paradygmatów

Aspekt	Odpowiedź grupy
Główny paradygmat	Hybryda OOP+Funkcyjny
Zastosowany język/i	C#
Uzasadnienie wyboru	Projekt wykorzystuje przede wszystkim paradygmat obiektowy z elementami podejścia funkcyjnego. OOP jest głównym paradygmatem, ponieważ domena składa się z obiektów posiadających stan i zachowanie: samolotu, systemów pokładowych, sensorów, komend, zdarzeń, stanów lotu i strategii pogody. Elementy funkcyjne są użyte tam, gdzie upraszczają przetwarzanie: w statycznych obliczeniach autopilota, LINQ, Option<T>, niemutowalnym SimulationConfig oraz przekazywaniu lambd do ActivateSystemCommand.

2.2. Uzasadnienie paradygmatu obiektowego (OOP) – jeśli zastosowany

Jeżeli projekt wykorzystuje OOP, odpowiedz na poniższe pytania. Dla każdego punktu podaj konkretny przykład z waszego kodu (nazwa klasy, metody, wzorca).

- **Enkapsulacja:** W systemie ukryto kluczowe dane i logikę fizyki lotu, chroniąc je przed niepożądaną modyfikacją z zewnątrz. Główna klasa **Aircraft** agreguje poszczególne podsystemy (np. **EngineSystem**, **FuelSystem**, **FlightData**), udostępniając do interakcji jedynie bezpieczne metody publiczne, takie jak **ApplyDamage()** czy **Update()**. Same stany wewnętrzne (jak **_systemsHealth** czy właściwość **_currentState**) są prywatne. Taki podział na odizolowane od siebie klasy idealnie odzwierciedla modułową budowę prawdziwego statku powietrznego, gdzie awaria np. hydrauliki jest enkapsulowana w dedykowanej klasie **HydraulicSystem** i nie modyfikuje bezpośrednio zmiennych w klasie silnika. Dzięki temu reszta systemu nie manipuluje dowolnie polami, tylko używa jasnych operacji domenowych.

- **Dziedziczenie i hierarchie:** projekt ma kilka hierarchii typów. **IAircraftState** jest kontraktem dla stanów lotu, a implementują go m.in. **GroundState**, **TaxiState**, **TakeOffState**, **ClimbState**, **CruiseState**, **LandingState**, **EmergencyState** i **CriticalState**. **AbstractAnomaly** dostarcza wspólne działanie anomalii, a konkretne klasy, np. **EngineFireAnomaly**, **FuelLeakAnomaly** czy **TurbulenceAnomaly**, implementują szczegóły. Sensory korzystają z klasy bazowej `Sensor`, a wyspecjalizowane typy, np. **AltitudeSensor**, **AirspeedSensor**, **FuelSensor**, rozszerzają jej zachowanie.
- **Polimorfizm:** **Aircraft** deleguje akcje do aktualnego **IAircraftState**, nie musi więc znać konkretnych reguł każdego stanu. **AnomalyEngine** przechowuje aktywne anomalie jako **List<IAomaly>** i wywołuje **Trigger**, **Update** oraz **Resolve** niezależnie od konkretnego typu. **EventBus** obsługuje różne implementacje **IFlightEventHandler**, np. **CascadeHandler**, **BlackBoxHandler**, **AlertSystemHandler**, **FlightLoggerHandler** i **StatisticsHandler**.
- **Wzorec projektowy:** W projekcie zastosowano kilka wzorców projektowych, ponieważ domena symulatora wymaga rozdzielenia odpowiedzialności i wymiennych zachowań.
 - **State** - użyty dla faz lotu. Naturalny wybór, bo samolot zachowuje się inaczej na ziemi, przy starcie, podczas wznoszenia, przelotu, lądowania i w sytuacji awaryjnej.
 - **Strategy** - użyty dla pogody i anomalii. Naturalny wybór, bo różne warunki pogodowe i awarie mają ten sam kontrakt, ale inne skutki.
 - **Observer** - użyty przez EventBus. Naturalny wybór, bo wiele modułów musi reagować na zdarzenia bez silnego sprzężenia z ich źródłem.
 - **Command** - użyty do sterowania z klawiatury. Naturalny wybór, bo akcje gracza można reprezentować jako obiekty, zapisywać w historii i częściowo cofać.
 - **Factory** - użyty do tworzenia samolotu, pogody i anomalii. Naturalny wybór, bo tworzenie tych obiektów zależy od konfiguracji lub kodu typu.
 - **Singleton** - użyty dla EventBus.Instance. Naturalny wybór w uproszczonej wersji projektu, bo system ma jedną centralną szynę zdarzeń.
 - **MVC / wariant Model-View-Controller** - model domenowy jest w Core, kontrolery w Controllers, a renderowanie w Views.
- **Zasady SOLID:**
 - **S - Single Responsibility Principle, zasada pojedynczej odpowiedzialności:** W projekcie widać to w podziale na klasy i foldery. InputHandler odpowiada za interpretację klawiatury i tworzenie komend. FlightController odpowiada za pętlę symulacji, zmianę widoku, pogodę, automatyczne fazy lotu i lądowanie. Aircraft Odpowiada za model domenowy samolotu. SensorSystem zarządza sensorami, a EventBus tylko rozsyła zdarzenia.

Przykład: gdy zmienia się sposób renderowania dashboardu, edytujemy klasy w Views, a nie Aircraft. Gdy dodajemy nową anomalię, edytujemy Core/Strategies/Anomalies i ewentualnie AnomalyFactory, a nie widok konsoli.

- **O - Open/Closed Principle, zasada otwarte/zamknięte:** W projekcie widac to w interfejsach `IWeatherStrategy`, `IAnomaly`, `IAircraftState` i `IFlightCommand`. Aby dodac nowy typ pogody, mozna utworzyc nowa klase implementujaca `IWeatherStrategy`. Aby dodac nowa anomalie, mozna utworzyc klase implementujaca `IAnomaly` albo dziedziczaca po `AbstractAnomaly`.
Nowa pogoda nie wymaga zmiany pętli symulacji w `FlightController`, bo kontroler wywołuje tylko `Apply`. W praktyce trzeba jeszcze dopisać przypadek do `WeatherFactory`, więc zasada `Open/Closed` jest spełniona częściowo, ale struktura projektu nadal mocno ułatwia rozszerzanie.
- **L - Liskov Substitution Principle, zasada podstawienia Liskov:** W projekcie każdy stan lotu może być traktowany jako `IAircraftState`. `Aircraft.TransitionTo(new LandingState())` czy `TransitionTo(new EmergencyState())` działają przez ten sam interfejs. Podobnie każda anomalia może być obsługiwana jako `IAnomaly` w `AnomalyEngine`. Pętla nie musi wiedzieć, czy obsługuje pożar silnika, wyciek paliwa, turbulencje czy awarie sensora. Każda klasa zachowuje kontrakt `IAnomaly`, czyli można ją podstawić w miejscu interfejsu.
- **I - Interface Segregation Principle, zasada segregacji interfejsów:** Projekt stosuje kilka mniejszych interfejsów: `IFlightCommand`, `IFlightEventHandler`, `IWeatherStrategy`, `IScreen`, `IWidget`, `ISensor`, `IAnomaly`. Dzięki temu np. handler zdarzeń musi znać tylko metodę `Handle`, a strategia pogody tylko `Name` i `Apply`.
Ograniczenie: `IAircraftState` ma dość szeroki interfejs, bo każdy stan musi mieć metody `TakeOff`, `Cruise`, `Descend`, `Land`, `HandleEmergency`, `Abort`, nawet jeśli część z nich w danym stanie nic nie robi. Jest to świadome uproszczenie wzorca `State`: dzięki temu `Aircraft` może delegować wszystkie akcje do aktualnego stanu jednolitym API.
- **D - Dependency Inversion Principle, zasada odwracania zależności:** W projekcie widać to w kilku miejscach. `FlightController` przyjmuje widoki jako `IScreen`, a nie jako konkretne klasy. `CommandHistory` operuje na `IFlightCommand`. `EventBus` przechowuje `IFlightEventHandler`. `AnomalyEngine` przechowuje `IAnomaly`. `Aircraft` deleguje zachowanie do `IAircraftState`. Dzięki temu można podmienić widok dashboardu lub kamery na inną implementację `IScreen`, bez zmiany głównego algorytmu sterowania symulacją. Zasada jest stosowana praktycznie, choć nie wszędzie idealnie, bo część obiektów nadal jest tworzona bezpośrednio w konstruktorach lub fabrykach.

2.3. Uzasadnienie paradygmatu funkcyjnego (FP) – jeśli zastosowany

Jeżeli projekt wykorzystuje paradygmat funkcyjny, odpowiedz na poniższe pytania.

- **Niemutowalność danych (immutability):**

Została zaimplementowana na dwa sposoby.

Po pierwsze, struktura monadyczna `Option<T>` została zadeklarowana jako typ wartościowy tylko do odczytu (*readonly struct*), a jej wewnętrzne pole przechowujące dane posiada modyfikator *private readonly T_value*. Oznacza to, że raz zamkniętej w monadzie wartości nie da się w żaden sposób zmodyfikować.

Po drugie, koncepcję niemutowalności rozszerzono na całą architekturę komunikacyjną systemu. Wszystkie klasy zdarzeń reprezentujące telemetrię i sytuacje awaryjne (np. `TelemetryTickEvent`

czy **EngineFireEvent**) pełnią rolę niemutowalnych obiektów transferu danych. Raz wygenerowane zdarzenie wędruje przez szynę **EventBus** do wielu odbiorców jednocześnie (np. do **BlackBoxHandler**). Dzięki temu, że te obiekty są niezmiennie, zyskujemy absolutną gwarancję, że żaden z asynchronicznie działających handlerów nie zmodyfikuje przypadkiem zawartości zdarzenia w trakcie jego przetwarzania.

Niemutowalność w naszym projekcie eliminuje błędy związane z efektami ubocznymi i ułatwia odtwarzanie historii lotu w testach.

- **Funkcje czyste (pure functions):**

Kluczowe pure functions zostały zaimplementowane w logice systemu Autopilota (**AutopilotSystem**) oraz w algorytmach wyliczających poprawki lotu (**AutopilotCalculator**). Metody takie jak: **CalculateNewPitch** czy **CalculateNewThrottle** przyjmują dane wejściowe wyłącznie jako argumenty i zwracają wynik liczbowy, bez żadnych efektów ubocznych (nie modyfikują niczego zewnętrznego).

Brak efektów ubocznych jest kluczowy z powodu Determinizmu (dla tych samych argumentów zawsze zwraca te same wartości) i łatwości testowania (skoro funkcje są odizolowane od stanu obiektu, to można je weryfikować przekazując im jedynie surowe wartości bez dodatkowych symulacji).

- **Funkcje wyższego rzędu (HOF):** Wykorzystaliśmy w projekcie zarówno własne funkcje wyższego rzędu, jak i te wbudowane w bibliotekę LINQ (odpowiedniki map, filter i reduce).

-W strukturze **Option<T>** zdefiniowaliśmy *IfPresent(Action<T> action)*. Przyjmuje ona inną funkcję (delegata) jako argument i wykonują ją bezpiecznie tylko wtedy, gdy w monadzie faktycznie znajdują się dane. Celem tej funkcji jest deklaratywne sterowanie przepływem programu i eliminacja imperatywnych instrukcji warunkowych.

-W klasie **StatisticHandler**, służącej do generowania raportów z lotu, potoki LINQ analizują historię zdarzeń (*FlightEvent*). Używamy tam metod wyższego rzędu np. *.Where()* (odpowiednik filter), *.Select()* (odpowiednik map), czy metod agregujących (odpowiednik reduce).

Zaimplementowaliśmy je w celu zastąpienia funkcji for/foreach instrukcjami deklaratywnymi. Powoduje to, że unikamy nadmiernej objętości kodu w projekcie i ułatwieniami w testowaniu (w testach weryfikujemy już gotowy potok transformacji).

- **Domknięcia (closures):** W naszym projekcie pojawiają się one naturalnie podczas korzystania z wyrażień lambda, głównie w kontekście budowania potoków LINQ i rejestrowania funkcji zwrotnych w szynie zdarzeń **EventBus**.

-W klasach analitycznych typu potoki wewnątrz **StatisticsHandler**, często filtrujemy historię lotu na podstawie dynamicznych parametrów przekazywanych do metody. Podczas definiowania takich wyrażień jak *.Where* nasza lambda przechwytuje (zamyka) zmienną *threshold* z otaczającego ją kontekstu metody. Dzięki temu delegat zapamiętuje wartość tej zmiennej na potrzeby późniejszej, leniwej ewaluacji.

-W **EventBus** natomiast podczas konfigurowania subskrypcji (np. w testach jednostkowych lub przy inicjalizacji handlerów), przekazujemy lambdy jako funkcje obsługujące zdarzenia. Często

wewnątrz tych lambd odwołujemy się do zmiennych zadeklarowanych wyżej w kodzie (np. do lokalnego licznika błędów lub flagi statusu). Funkcja lambda tworzy domknięcie, "zamykając" tę zmienną w swoim własnym stanie. Pozwala to asynchronicznemu delegatowi na dostęp do zmiennej w momencie wystąpienia zdarzenia w przyszłości, nawet jeśli metoda, która tę zmienną utworzyła, już dawno zakończyła swoje działanie.

- **Generatory / leniwa ewaluacja:** Tak, projekt korzysta z generatorów. Zostały one zaimplementowane poprzez interfejs **IEnumerable<T>** i zapytań LINQ w modułach przetwarzających historię lotu (**StatisticHandler/BlackBoxHandler**).

Kiedy budujemy potok danych (np. filtracja historii lotu podczas szukania awarii), operacje te nie są wykonywane natychmiastowo. Zamiast alokować nowe listy w pamięci na każdym etapie transformacji, kompilator tworzy w tle generator. Następnie zapytanie jest leniwie ewaluowane - elementy strumienia są generowane i przetwarzane, dopiero gdy tego zażądamy (np. gdy chcemy wyświetlić wynik na ekranie).

Dają nam one dużą oszczędność pamięci - Leniwa ewaluacja zapobiega tworzeniu się ogromnych, tymczasowych kolekcji w pamięci RAM podczas ich filtrowania i pozwalają obsługiwać wielogodzinne sekwencje bez wczytywania wszystkiego do pamięci operacyjnej na raz.

- **Dekoratory:** Zaimplementowaliśmy je w sposób funkcyjny oraz obiektowy (wzorzec Decorator), polegający na "opakowywaniu" (wrapping) funkcji bazowych lub całych interfejsów za pomocą funkcji wyższego rzędu i klas pośredniczących.

W naszym projekcie zjawisko dekorowania wykorzystano w systemie obsługi poleceń pilota (moduł **CommandHistory** i interfejs **IFlightCommand**) oraz przy wywoływaniu zdarzeń w **EventBus**.

Dekorator wzbogaca czystą funkcję zmieniającą stan systemu o automatyczne wygenerowanie wpisu do systemu logów (**FlightLogger**) oraz czarnej skrzynki. Dzięki temu funkcja bazowa (np. zmieniająca wychylenie kłap) pozostaje "czysta" i nie musi być zanieczyszczona kodem odpowiedzialnym za logowanie.

- **Potoki przetwarzania (pipelines):** Zostały one zaimplementowane dzięki bibliotece LINQ. Ich główne zastosowanie znajduje się w **StatisticsHandler**.

Transformacje są łączone za pomocą metody łańcuchowej bez użycia jakichkolwiek zmiennych pośrednich. Każdy etap potoku (np. **Where()**) jest czystą funkcją, która przyjmuje niemutowalny strumień danych w postaci interfejsu **IEnumerable<T>**, aplikuje na nim swoją logikę, a następnie leniwie zwraca nowy, zmodyfikowany strumień **IEnumerable<T>**, który natychmiast staje się wejściem dla kolejnej funkcji w łańcuchu.

2.4. Uzasadnienie podejścia hybrydowego (jeśli dotyczy)

Jeśli projekt łączy oba paradygmaty, opisz, w jaki sposób podział odpowiedzialności między OOP a FP jest uzasadniony. Odpowiedz na pytania:

- Które warstwy aplikacji (np. logika biznesowa, logika przetwarzania, I/O) są realizowane obiektowo, a które funkcyjnie?
 - **Warstwa obiektowa (OOP)** realizuje modelowanie domeny problemu oraz zarządzanie ogólnym cyklem Życia symulacji. Obiektowo zrealizowano architekturę szkieletową: hierarchię podsystemów (np. **EngineSystem**, **HydraulicSystem**), zarządzanie wewnętrznym cyklem maszyny stanów (wzorzec State w postaci np. **CruiseState**, **LandingState**) oraz abstrakcje polimorficzne dla strategii awarii i pogody (IAnomaly, IWeatherStrategy).
 - **Warstwa funkcyjna (FP)** obsługuje logikę obliczeniową, niezawodność komunikacji oraz bezpieczeństwo typów. Funkcyjnie zrealizowano warstwę transportową systemu zdarzeń (komunikaty jako niemutowalne rekordy w **EwentBus**), warstwę matematyczną i kalkulacyjną (Czyste Funkcje w **AutopilotSystem**, które przeliczają wektory bez wywoływania efektów ubocznych), a także mechanizmy kontroli przepływu oparte na typach uogólnionych (własna implementacja monady **Option<T>** zastępująca konwencję zwracania mutowalnych nulli lub flag -1.0).
- W jaki sposób paradygmaty się uzupełniają, a nie kolidują?

Odpowiedź: Warstwa domenowa jest obiektowa: samolot, systemy, stany, anomalie i komendy mają własny stan oraz zachowanie. Elementy funkcyjne pojawiają się w miejscach, gdzie potrzebne są proste transformacje danych, obliczenia bez efektów ubocznych albo bezpieczna obsługa braku wartości. Paradygmaty się uzupełniają: OOP porządkuje model domenowy, a FP zmniejsza ryzyko błędów w obliczeniach i filtracji kolekcji. Dzięki temu nie ma problemu jak w złożonych symulacjach w pełni stosujących OOP, z chaotycznym kodem.

Przykładowo: obiektowy **SensorSystem** po wykryciu awarii nie modyfikuje bezpośrednio klas **DamageModel**. Zamiast tego publikuje niemutowalne zdarzenie (FP), które jest przetwarzane przez czyste funkcje w handlerach. OOP tworzy architekturę, a FP gwarantuje przewidywalność logiki.
- Podaj analogię do przykładu z zajęć (wzorzec Strategia + potoki przetwarzania) i wyjaśnij, jak wasza hybryda jest do niego podobna lub czym się różni.

Odpowiedź: W porównaniu do przykładu Strategia + pipeline z zajęć, projekt jest podobny przez użycie strategii pogody/anomalii i przetwarzanie danych telemetrycznych, ale różni się tym, że centralnym problemem jest symulacja stanu w czasie rzeczywistym, więc pełne FP nie byłoby naturalne.



WSKAZÓWKA

Sekcja 2 jest oceniana szczegółowo. Zwykle stwierdzenie „wybraliśmy OOP, bo znamy Javę” nie jest uzasadnieniem. Uzasadnienie musi wynikać z analizy domeny problemu.

3. Architektura i projekt systemu

3.1. Diagram architektury wysokiego poziomu

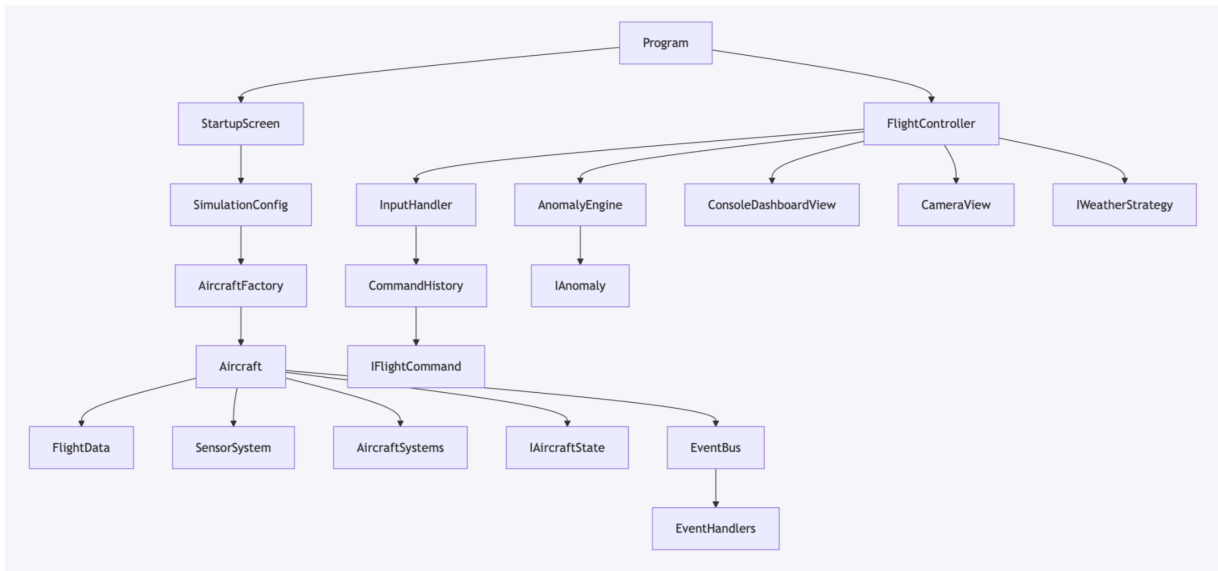
Umieść diagram (UML lub własny zapis graficzny) ilustrujący strukturę systemu. Diagram musi pokazywać główne komponenty / moduły / klasy i relacje między nimi. Poniżej diagramu dodaj opis (3–5 zdań).

AeroSimulator:

- **Controllers:**
 - AnomalyEngine
 - FlightController
 - InputHandler
- **Core:**
 - **Aircraft:**
 - **Enums:**
 - FireLocation
 - **Sensors:**
 - AirspeedSensor
 - AltitudeSensor
 - EngineSensor
 - EngineTempSensor
 - FuelSensor
 - HydraulicSensor
 - ISensor (*Interfejs*)
 - Sensor
 - SensorSystem
 - **Systems:**
 - AutopilotSystem
 - DamageModel
 - ElectricalSystem
 - EngineSystem
 - EngineUnit
 - FuelSystem
 - HydraulicSystem
 - IAircraftSystem (*Interfejs*)
 - NavigationSystem
 - WeatherSystem
 - WingSystem
 - Aircraft
 - FlightData
- **Commands:**
 - ActivateSystemCommand
 - CommandHistory
 - EmergencyDeclareCommand
 - GoAroundCommand
 - IFlightCommand (*Interfejs*)
 - ResolveAnomalyCommand
 - SetAltitudeCommand
 - SetHeadingCommand
 - SetPitchCommand

- SetThrottleCommand
 - ToggleAutopilotCommand
- **Common:**
 - Option
- **Events:**
 - **Handlers:**
 - AlertBufferHandler
 - AlertSystemHandler
 - BlackBoxHandler
 - CascadeHandler
 - FlightLoggerHandler
 - EventBus
 - IFlightEventHandler
- **States:**
 - ClimbState
 - CriticalState
 - CruiseState
 - DescentState
 - EmergencyState
 - GroundState
 - HoldingState
 - IAircraftState
 - LandingState
 - TakeOffState
 - TaxiState
- **Strategies:**
 - **Anomalies:**
 - AbstractAnomaly
 - BirdStrikeAnomaly
 - DecompressionAnomaly
 - ElectricalFailureAnomaly
 - EngineFailureAnomaly
 - EngineFireAnomaly
 - FuelLeakAnomaly
 - HydraulicFailureAnomaly
 - IAnomaly (*Interfejs*)
 - IcingAnomaly
 - MicroburstAnomaly
 - RunwayIncursionAnomaly
 - SensorFailureAnomaly
 - TurbulenceAnomaly
 - WingFireAnomaly
 - **Weather:**
 - ClearSkiesStrategy
 - CrosswindStrategy

- FogStrategy
 - IcingConditionsStrategy
 - IWeatherStrategy (*Interfejs*)
 - ThunderstormStrategy
 - WindShearStrategy
- Infrastructure:
 - FlightLogger
 - IScreen
 - IWidget
- Views:
 - Components:
 - AlertWidget
 - ConsoleDashboardView
 - FlightDataWidget
 - HorizonWidget
 - LegendWidget
 - MenuHeaderWidget
 - SensorsPanelWidget
 - SettingsSummaryWidget
 - StartupScreen
 - SystemsPanelWidget
 - BlackboxReadoutView
 - CameraView
 - IntroSplashScreen
- Program - uruchomienie



Opis: Program uruchamia ekrany startowe, tworzy konfigurację i buduje model samolotu przez AircraftFactory. FlightController jest centralnym kontrolerem pętli symulacji: aktualizuje pogodę, model lotu, anomalie, wejście użytkownika i widok. Aircraft jest głównym modelem domenowym, który agreguje systemy pokładowe, sensory, dane lotu, model uszkodzeń i aktualny stan. Komunikacja między modułami odbywa się przez EventBus, dzięki czemu logowanie, alerty, black box i kaskady awarii nie muszą być bezpośrednio połączone z kontrolerem.

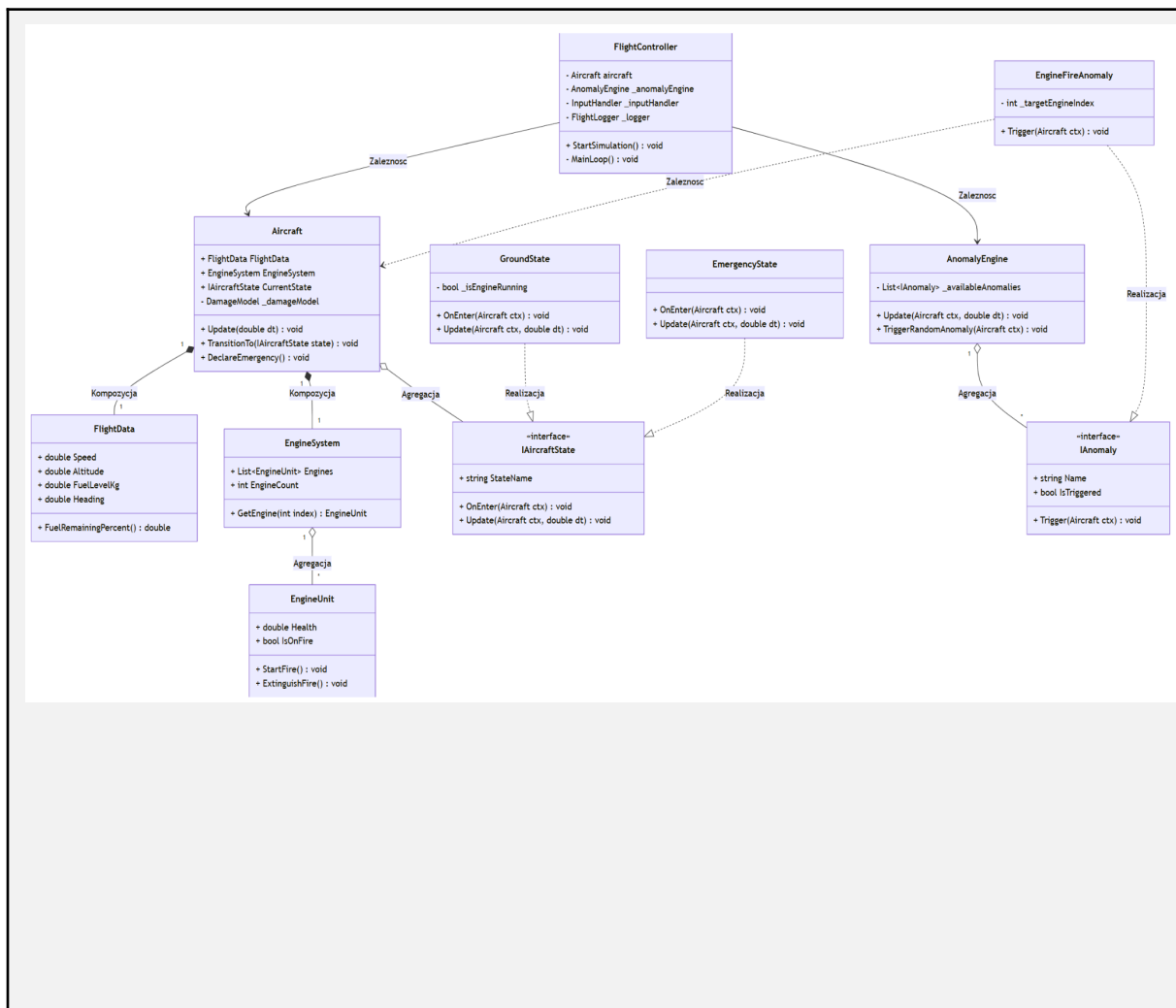
3.2. Opis głównych komponentów / modułów

Dla każdego głównego komponentu uzupełnij tabelę:

Nazwa komponentu/modułu	Odpowiedzialność	Paradygmat
Program	Uruchomienie aplikacji, konfiguracja konsoli, start symulacji i raport końcowy	proceduralny + OOP
Controllers	Pętla symulacji, wejście użytkownika, zarządzanie anomaliami	OOP
Core/Aircraft	Model samolotu, dane lotu, systemy, sensory, uszkodzenia	OOP
Core/States	Fazy lotu i reguły przejść między nimi	OOP, State
Core/Strategies	Pogoda i anomalie jako wymienne strategie	OOP, Strategy
Core/Events	Zdarzenia, event bus, alerty, black box, logowanie	OOP, Observer
Core/Commands	Komendy sterowania i historia undo	OOP + FP (lambdy)
Views	Renderowanie dashboardu, kamery, splash screen, raportu	OOP, MVC
Infrastructure	Fabryki, presety danych, konfiguracja symulacji	OOP + FP (dane niemutowalne)

3.3. Diagram klas UML (dla komponentów OOP)

Przedstaw diagram klas UML dla głównych klas aplikacji. Diagram musi zawierać: nazwy klas, pola z typami, metody publiczne, relacje (dziedziczenie, kompozycja, agregacja, zależność). Stosuj standardową notację UML.



Opis:

Aircraft (centralny punkt symulacji, uruchamianie innych plików): Pełni funkcję korzenia agregacji. Klasa ta enkapsuluje złożoną logikę i ukrywa detale implementacyjne systemów wewnętrznych przed resztą aplikacji. Zewnętrzne komponenty nie modyfikują bezpośrednio poszczególnych podzespołów – komunikacja odbywa się wyłącznie poprzez publiczne metody klasy Aircraft, co gwarantuje spójność i bezpieczeństwo stanu całego samolotu.

FlightController (pętla symulacji zarządzanie czasem i delegowanie sygnałów wejściowych) : Realizuje Separację Obaw. Zgodnie z Zasadą Pojedynczej Odpowiedzialności (SRP), logika domeny (latający samolot) jest całkowicie oddzielona od logiki sterowania i interfejsu. Dzięki temu klasa Aircraft nie wie nic o klawiaturze czy konsoli, co w przyszłości umożliwia bezbolesną migrację symulatora na inny interfejs graficzny

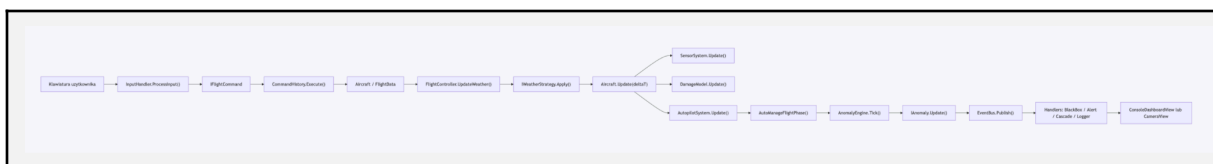
FlightData i **EngineControlSystem** (przechowuje prędkość, wysokość paliwo; zarządza jednostkami napędowymi) : Aplikacja mocno stawia na zasadę "Kompozycji ponad dziedziczenie". Zamiast tworzyć monolityczną, gigantyczną klasę samolotu zawierającą setki zmiennych, Aircraft komponuje w sobie mniejsze, wysoce wyspecjalizowane obiekty. Zwiększa to czytelność, zapobiega powielaniu kodu i ułatwia testowanie jednostkowe (Unit Testing) każdego systemu z osobna.

IAircraftStates (definiuje zachowanie samolotu i fizyki) : Wykorzystanie wzorca projektowego Stan (State Pattern) i polimorfizmu. Wyeliminowano dzięki temu rozbudowane, trudne w utrzymaniu drabinki instrukcji warunkowych (if/switch) wewnątrz głównej pętli. System ściśle przestrzega Zasady Otwarte-Zamknięte (Open/Closed Principle) – dodanie nowej fazy lotu wymaga jedynie utworzenia nowej klasy, bez konieczności modyfikowania istniejącego kodu.

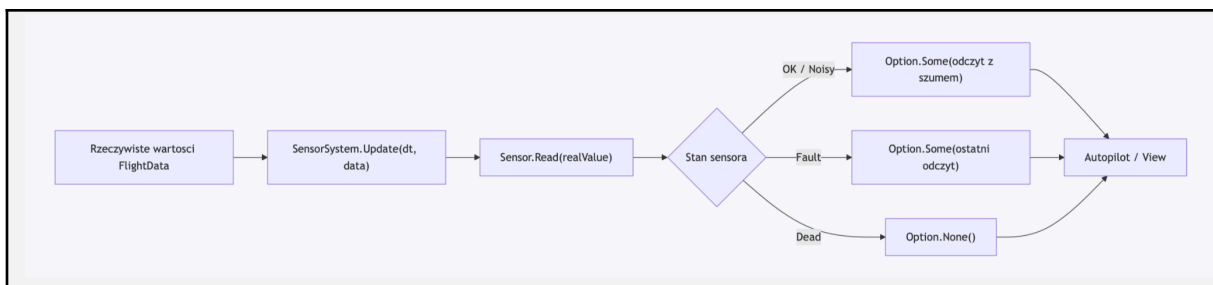
IAnomaly i **AnomalyEngine** (losowe generowanie anomalii): Moduł ten opiera się na Zasadzie Odwrócenia Zależności (Dependency Inversion). Silnik zarządza awariami poprzez abstrakcję (interfejs IAnomaly), a nie poprzez ich konkretne implementacje. Skutkuje to całkowitą izolacją zdarzeń – silnik nie musi wiedzieć, jak działa pożar czy awaria hydrauliki, obchodzi go jedynie to, że obiekt spełnia kontrakt interfejsu i posiada metodę Trigger().

3.4. Diagram przepływu danych / potok przetwarzania (dla komponentów FP)

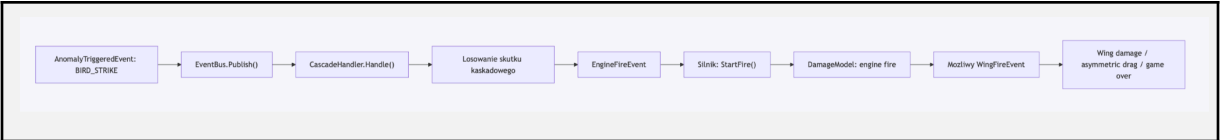
Dla każdego potoku przetwarzania (pipeline) opisz sekwencję transformacji: dane wejściowe → funkcja 1 → funkcja 2 → ... → wynik. Możesz użyć schematu blokowego lub pseudokodu z zastosowanymi HOF.



Opis: dane wejściowe pochodzą z klawiatury. InputHandler tłumaczy je na obiekty komend, np. SetThrottleCommand, SetPitchCommand, ToggleAutopilotCommand albo ActivateSystemCommand. Komenda zmienia stan modelu Aircraft albo jego podsystemów. Następnie FlightController wykonuje jeden krok symulacji: aktualizuje pogodę, fizykę lotu, paliwo, sensory, autopilota, fazę lotu i aktywne anomalie. Jeżeli w trakcie aktualizacji pojawi się ważne zdarzenie, trafia ono do EventBus, a dalej do handlerów odpowiedzialnych za alerty, black box, logowanie i awarie kaskadowe. Wynikiem potoku jest zaktualizowany model oraz odświeżony widok konsolowy.



Opis: FlightData przechowuje rzeczywiste parametry lotu, ale kokpit i autopilot nie muszą czytać ich bezpośrednio. Odczyt przechodzi przez SensorSystem, który używa konkretnych sensorów. Sensor może zwrócić poprawną wartość, wartość z szumem, zatrzymany ostatni odczyt albo brak wartości. Brak wartości jest reprezentowany przez Option.None(), co ogranicza ryzyko błędów null.



Opis: przykład pokazuje, że anomalie nie są izolowane. Zdarzenie BIRD_STRIKE może zostać przechwycone przez CascadeHandler, który z pewnym prawdopodobieństwem publikuje EngineFireEvent. Pożar silnika może uszkodzić silnik, zabić sensor RPM, rozprzestrzeniać się na skrzydło i doprowadzić do trudniejszego sterowania. To jest potok zdarzeniowy, w którym wynik jednego etapu staje się wejściem kolejnego.

4. Opis implementacji

4.1. Przegląd technologii i narzędzi

Wypełnij poniższą tabelę:

Technologia / Biblioteka	Wersja	Rola w projekcie
C#	zgodny z projektem .NET	język implementacji
.NET	net10, w plikach .csproj	Platforma uruchomieniowa programu
xUnit	2.9.3	testy jednostkowe i integracyjne
FluentAssertions	8.10.0	czytelniejsze asercje w testach
konsola systemowa	standard library	interfejs użytkownika i ender dashboardu

4.2. Kluczowe fragmenty kodu z omówieniem

Przedstaw 3–6 najważniejszych fragmentów kodu (listing). Każdy listing musi być poprzedzony tytułem i opisem (co robi, dlaczego jest ważny z perspektywy paradygmatu), a po listingu zamieść analizę (3–5 zdań).

Listing 1: Implementacja wzorca State dla faz lotu

Ten fragment pokazuje, jak projekt modeluje fazy lotu jako osobne klasy implementujące wspólny interfejs. Jest to ważne z perspektywy OOP, ponieważ zachowanie samolotu zależy od aktualnego stanu, ale Aircraft nie musi posiadać jednej ogromnej instrukcji warunkowej.

```

public interface IAircraftState
{
    string StateName { get; }
    string StateDescription { get; }
    ConsoleColor StateColor { get; }
    IReadOnlyList<string> AllowedActions { get; }

    void TakeOff(Aircraft ctx);
    void Cruise(Aircraft ctx);
    void Descend(Aircraft ctx);
    void Land(Aircraft ctx);
    void HandleEmergency(Aircraft ctx);
    void Abort(Aircraft ctx);
    void Update(Aircraft ctx, double deltaT);
    void OnEnter(Aircraft ctx);
    void OnExit(Aircraft ctx);
}

public class TakeOffState : IAircraftState
{
    public string StateName => "TAKEOFF";
    public string StateDescription => "Full throttle takeoff run on the runway.";
    public ConsoleColor StateColor => ConsoleColor.Yellow;
    public IReadOnlyList<string> AllowedActions => new List<string> { "Abort", "HandleEmergency" };

    private bool _hasRotated;

    public void OnEnter(Aircraft ctx)
    {
        ctx.FlightData.Throttle = 1.0;
        _hasRotated = false;
    }

    public void Update(Aircraft ctx, double deltaT)
    {
        ctx.FlightData.Speed += 25.0 * deltaT;

        if (ctx.FlightData.Speed >= ctx.Config.Aircraft.VRSpeedKts && !_hasRotated)
        {
            ctx.FlightData.PitchAngleDeg = 7.5;
            _hasRotated = true;
        }

        if (ctx.FlightData.Altitude >= 1500.0)
        {
            ctx.TransitionTo(new ClimbState());
        }
    }
}

```

```

    }
}
}

```

Analiza: `IAircraftState` jest abstrakcją, a `TakeOffState` konkretna implementacja stanu. Dzięki polimorfizmowi samolot może wywołać `CurrentState.Update(...)`, nie znając szczegółów aktualnej fazy. Enkapsulacja polega na tym, że logika startu, rotacji i przejścia do wznoszenia znajduje się w klasie odpowiedzialnej za start. Wzorzec State poprawia czytelność, bo reguły dla poszczególnych faz lotu nie są wymieszane w jednej metodzie.

Listing 2: Implementacja wzorca Strategy i Template Method dla anomalii

Ten fragment pokazuje, jak wszystkie anomalie mają wspólny kontrakt, a klasa abstrakcyjna dostarcza wspólny algorytm aktywacji i aktualizacji. Konkretnie anomalie implementują tylko szczegóły: co dzieje się przy uruchomieniu, w czasie trwania i przy rozwiązaniu problemu.

```

public interface IAnomaly
{
    string AnomalyName { get; }
    string Description { get; }
    Severity Level { get; }
    double Probability { get; }
    bool IsActive { get; }
    bool CanBeResolved { get; }

    void Trigger(Aircraft ctx, FlightData data);
    void Update(Aircraft ctx, FlightData data, double deltaT);
    bool Resolve(Aircraft ctx);
}

public abstract class AbstractAnomaly : IAnomaly
{
    protected readonly Random _rng = new();
    protected double _activeDuration;
    protected bool _isActive;

    public void Trigger(Aircraft ctx, FlightData data)
    {
        if (!_isActive) return;
        _isActive = true;
        _activeDuration = 0;
        OnTrigger(ctx, data);
        ctx.Publish(new AnomalyTriggeredEvent(AnomalyName, Level, GetWarningMessage()));
    }

    public void Update(Aircraft ctx, FlightData data, double deltaT)
    {
        if (!_isActive) return;

```

```

        _activeDuration += deltaT;
        OnUpdate(ctx, data, deltaT);
    }

    protected abstract void OnTrigger(Aircraft ctx, FlightData data);
    protected abstract void OnUpdate(Aircraft ctx, FlightData data, double deltaT);
    protected abstract bool OnResolve(Aircraft ctx);
}

```

Analiza: IAnomaly jest strategią opisującą wymienne zachowanie awarii. AbstractAnomaly zawiera wspólny szkielet algorytmu, więc przypomina też wzorzec Template Method: metoda Trigger zawsze ustawia aktywność i publikuje zdarzenie, ale szczegóły są delegowane do OnTriggerer. Konkretnie klasy, np. EngineFireAnomaly, dopisują efekty specyficzne dla pożaru silnika. Dzięki temu AnomalyEngine może przechowywać wszystkie awarie w jednej kolekcji List<IAnomaly> i aktualizować je polimorficznie.

Listing 3: Command dla wejścia użytkowników i historii komend

Ten fragment pokazuje, jak akcje gracza zostały opakowane w obiekty komend. Jest to ważne, bo wejście z klawiatury nie modyfikuje bezpośrednio całego modelu, tylko tworzy konkretną komendę, która wie, jak wykonać akcje.

```

public interface IFlightCommand
{
    string Name { get; }
    string Details { get; }
    void Execute(AircraftModel aircraft);
    void Undo(AircraftModel aircraft);
}

public class SetThrottleCommand : IFlightCommand
{
    private readonly double _delta;
    private double _previousThrottle;

    public string Name => "SetThrottle";
    public string Details => _delta >= 0 ? "Throttle increased" : "Throttle decreased";

    public void Execute(AircraftModel aircraft)
    {
        _previousThrottle = aircraft.FlightData.Throttle;
        aircraft.FlightData.Throttle = Math.Clamp(_previousThrottle + _delta, 0.0, 1.0);
    }

    public void Undo(AircraftModel aircraft)
    {
        aircraft.FlightData.Throttle = _previousThrottle;
    }
}

```

```
}
```

Analiza: IFlightCommand definiuje wspólny kontrakt dla akcji użytkownika. SetThrottleCommand przechowuje poprzednią wartość przepustnicy, dlatego może wykonać Undo. To jest klasyczny przykład enkapsulacji operacji jako obiektu. CommandHistory dodatkowo zapisuje wykonane komendy, co pozwala rozbudować projekt np. o pełny replay lub bardziej zaawansowane cofanie akcji.

Listing 4: EventBus jako wzorzec Observer

Ten fragment pokazuje centralną szynę zdarzeń. Komponent publikujący zdarzenie nie musi wiedzieć, kto je odbierze. Dzięki temu alerty, black box, statystyki i kaskady awarii mogą reagować niezależnie.

```
public class EventBus
{
    private static readonly Lazy<EventBus> _instance = new(() => new EventBus());
    public static EventBus Instance => _instance.Value;

    private readonly List<IFlightEventHandler> _handlers = new();
    private readonly object _lock = new();

    public void Subscribe(IFlightEventHandler handler)
    {
        if (handler == null) return;
        lock (_lock)
        {
            if (!_handlers.Contains(handler))
            {
                _handlers.Add(handler);
            }
        }
    }

    public void Publish(FlightEvent evt)
    {
        if (evt == null) return;

        List<IFlightEventHandler> activeHandlers;
        lock (_lock)
        {
            activeHandlers = new List<IFlightEventHandler>(_handlers);
        }

        foreach (var handler in activeHandlers)
        {
            handler.Handle(evt);
        }
    }
}
```

Analiza: EventBus pełni rolę Publishera, a klasy implementujące IFlightEventHandler pełnią rolę obserwatorów. Publish rozsyła zdarzenie do wszystkich aktualnych handlerów. Zastosowanie listy handlerów i interfejsu daje polimorfizm: BlackBoxHandler, CascadeHandler i AlertSystemHandler są traktowane jednolicie. Dodatkowo Lazy<EventBus> realizuje Singleton, czyli jedna wspólna instancja szyny zdarzeń w aplikacji.

4.3. Wzorce projektowe – szczegółowy opis (jeśli zastosowane)

Dla każdego zastosowanego wzorca projektowego wypełnij poniższy schemat:

- **Nazwa wzorca:** [np. Strategia, Obserwator, Dekorator, Fabryka, Singleton, inny]
- **Motywacja:** [Dlaczego ten wzorec był potrzebny? Jaki problem projektowy rozwiązuje?]
- **Uczestnicy:** [np. Które klasy/interfejsy pełnią rolę Context, Strategy, ConcreteStrategy itd.?]
- **Połączenie z FP (jeśli hybryda):** [Np. strategie są implementowane jako czyste funkcje / pipeline]
- **Korzyści w tym projekcie:** [Co zyskaliście stosując wzorec zamiast rozwiązania ad hoc?]

Wzorec: State

Motywacja: samolot zachowuje się inaczej w zależności od fazy lotu. Inne reguły obowiązują na ziemi, inne podczas startu, inne podczas przelotu, lądowania albo awarii. Gdyby wszystko było w jednej klasie, powstałby bardzo duży switch po stanie lotu.

Uczestnicy: Aircraft pełni rolę Context. IAircraftState pełni rolę State. Konkretnie stany to m.in. GroundState, TaxiState, TakeOffState, ClimbState, CruiseState, DescentState, LandingState, EmergencyState, CriticalState.

Połączenie z FP: niewielkie. Wzorec jest typowo obiektowy, ale poszczególne stany korzystają z prostych obliczeń na danych liczbowych.

Korzyści w tym projekcie: kod faz lotu jest rozdzielony i czytelny. Dodanie nowej fazy, np. HoldingState, nie wymaga przebudowania całej klasy Aircraft. Projekt lepiej pokazuje polimorfizm, bo Aircraft deleguje akcje do aktualnego IAircraftState.

Wzorec: Strategy

Motywacja: pogoda i anomalie są wymiennymi zachowaniami. Każda pogoda ma metode Apply, ale ClearSkiesStrategy, ThunderstormStrategy, FogStrategy i WindShearStrategy wpływają na samolot inaczej. Podobnie anomalie mają wspólne operacje Trigger, Update, Resolve, ale każda awaria realizuje je inaczej.

Uczestnicy: dla pogody Contextem jest FlightController, strategia to IWeatherStrategy, a konkretne strategie to ClearSkiesStrategy, ThunderstormStrategy, FogStrategy, CrosswindStrategy, IcingConditionsStrategy, WindShearStrategy. Dla anomalii Contextem jest AnomalyEngine, strategia to IAnomaly, a konkretne strategie to np. EngineFireAnomaly, FuelLeakAnomaly, TurbulenceAnomaly, SensorFailureAnomaly.

Połączenie z FP: strategie działają jak wymienne funkcje modyfikujące stan symulacji w danym kroku czasu. Dodatkowo wybór aktywnych anomalii i sensorów korzysta z LINQ, np. Where, Any, OfType.

Korzyści w tym projekcie: można dodawać nowe warunki pogodowe i anomalie bez przepisywania głównej petli. Kod jest bardziej modułowy, a każda strategia ma jasno wydzieloną odpowiedzialność.

Wzorzec: Observer

Motywacja: wiele części systemu musi reagować na zdarzenia, ale nie powinny być ze sobą bezpośrednio połączone. Na przykład zdarzenie awarii powinno pojawić się w alertach, black boxie, logach i czasem uruchomić kaskadę.

Uczestnicy: EventBus jest Publisherem/Subjectem. FlightEvent i klasy pochodne są zdarzeniami. IFlightEventHandler jest interfejsem obserwatora. Konkretni obserwatorzy to CascadeHandler, BlackBoxHandler, AlertSystemHandler, FlightLoggerHandler, AlertBufferHandler, StatisticsHandler.

Połączenie z FP: EventBus.Publish działa jak potok rozesłania zdarzenia do listy handlerów. Sam mechanizm jest obiektowy, ale ma charakter przetwarzania strumienia zdarzeń.

Korzyści w tym projekcie: moduł publikujący zdarzenie nie musi wiedzieć, kto je odbierze. Dzięki temu łatwo było dodać black box, alerty i kaskady awarii bez łączenia wszystkiego bezpośrednimi zależnościami.

Wzorzec: Command

Motywacja: wejście użytkownika powinno być zamieniane na akcje w sposób uporządkowany. Zamiast wykonywać całą logikę w switch klawiatury, projekt tworzy obiekty komend.

Uczestnicy: InputHandler pełni rolę Invokera, IFlightCommand jest interfejsem Command, konkretne komendy to SetThrottleCommand, SetHeadingCommand, SetPitchCommand, ToggleAutopilotCommand, ResolveAnomalyCommand, GoAroundCommand, EmergencyDeclareCommand, ActivateSystemCommand. Aircraft jest Receiverem, czyli obiektem, na którym wykonywana jest akcja. CommandHistory przechowuje historię.

Połączenie z FP: ActivateSystemCommand przyjmuje Action<Aircraft>, czyli funkcję przekazaną jako argument. Dzięki temu wiele prostych akcji systemowych można stworzyć bez osobnej klasy dla każdej z nich.

Korzyści w tym projekcie: sterowanie jest czytelniejsze i łatwiej rozszerzalne. Część komend można cofać przez Undo. Historia komend może być użyta do debugowania albo przyszłego odtwarzania akcji.

Wzorzec: Factory

Motywacja: tworzenie obiektów zależy od konfiguracji albo tekstowego kodu typu. Fabryki ukrywają szczegóły konstrukcji i centralizują proces tworzenia.

Uczestnicy: AircraftFactory tworzy Aircraft na podstawie SimulationConfig. AnomalyFactory tworzy konkretne IAnomaly na podstawie kodu, np. "ENGINE_FIRE" albo "FUEL_LEAK". WeatherFactory tworzy konkretne IWeatherStrategy na podstawie typu pogody.

Połączenie z FP: fabryki same nie są funkcyjne, ale zwracają obiekty implementujące wspólne kontrakty, które potem są używane jak wymienne zachowania w potoku symulacji.

Korzyści w tym projekcie: kod tworzący obiekty nie jest rozproszony po całej aplikacji. Jeśli dodajemy nową anomalię, wiadomo, gdzie wpisać sposób jej utworzenia. Konfiguracja startowa pozostaje czytelna.

Wzorzec: Singleton

Motywacja: w projekcie potrzebna jest jedna centralna szyna zdarzeń, do której mogą podpinac się handlers. Dlatego EventBus ma statyczną instancję Instance.

Uczestnicy: EventBus jest Singletonem. Używa Lazy<EventBus>, aby tworzyć instancje dopiero przy pierwszym użyciu.

Połączenie z FP: brak bezpośredniego połączenia. Jest to wzorzec obiektowy i infrastrukturalny.

Korzyści w tym projekcie: wszystkie komponenty publikują zdarzenia do tego samego miejsca. Upraszcza to architekturę małego projektu. Ograniczeniem jest globalny stan, dlatego w testach konstruktor Aircraft czyści handlers przez _eventBus.ClearHandlers().

Wzorzec: MVC / Model-View-Controller

Motywacja: logika symulacji nie powinna być wymieszana z wyświetlaniem konsolowym ani z obsługą klawiatury.

Uczestnicy: Model to Core, szczególnie Aircraft, FlightData, systemy, sensory, stany i anomalie. Controller to FlightController, InputHandler i AnomalyEngine. View to ConsoleDashboardView, CameraView, StartupScreen, FlightReportView, BlackBoxReadoutView oraz widgety w Views/Components.

Połączenie z FP: widoki głównie odczytują dane i renderują je, natomiast kontrolery wykonują potok aktualizacji. Nie jest to stricte FP, ale separacja danych, sterowania i prezentacji ułatwia testowanie funkcji domenowych.

Korzyści w tym projekcie: można zmienić sposób wyświetlania bez zmiany modelu samolotu. Kod jest łatwiejszy do czytania, bo wiadomo, gdzie szukać logiki symulacji, a gdzie renderowania.

5. Testowanie (mile widziane)

5.1. Strategia testowania

Opisz podejście do testowania: jakie rodzaje testów zostały zastosowane, jakie narzędzia (pytest, JUnit, Google Test, unittest itp.), i jak testy są zorganizowane.

- **Testy jednostkowe (unit tests):** Testy logiki systemów pokładowych (*AutopilotCalculator* czy potoki w *StatisticHandler*). Nasz projekt posiada kilkanaście takich testów. Weryfikują one poprawność i determinizm algorytmów sterujących.
- **Testy integracyjne:** czy testowano współpracę między modułami? Tak. Plik *AircraftIntegrationTests.cs* sprawdza współpracę konfiguracji samolotu, *Aircraft*, *FlightData*, *SensorSystem*, stanów lotu i anomalii. Testowane są m.in. przejścia z *TaxiState* do *TakeOffState*, przejście ze startu do *ClimbState*, wykrycie niskiego poziomu paliwa w *CruiseState* i aktywacja *EngineFailureAnomaly*.
- **Testy end-to-end:** czy testowano kompletny przepływ od wejścia do wyjścia? Nie ma pełnych testów end-to-end obejmujących rzeczywiste wejście z klawiatury, pętle konsolowa i renderowanie dashboardu. Jest to zrozumiałe, ponieważ aplikacja jest interaktywna i konsolowa, a testowanie E2E takiego programu wymaga osobnej infrastruktury do symulowania klawiszy i przechwytywania wyjścia konsoli. Zamiast tego projekt testuje najważniejszą logikę domenową i integrację modelu.
- **Pokrycie kodu (code coverage):** Szacujemy pokrycie na około 70%-80% dla krytycznej warstwy logiki biznesowej (moduły w folderze *Core*). Testami jednostkowymi i integracyjnymi objęto najważniejsze klasy dziedziczne, takie jak maszyna stanów (*Aircraft*), kluczowe systemy pokładowe (*AutopilotSystem*, *FuelSystem*, *EngineSystem*) oraz analitykę (*StatisticsHandler*). Z pomiaru pokrycia celowo wykluczono warstwę prezentacji (folder *Views* oraz powiązane z nią widżety) oraz punkty wejścia do aplikacji, ponieważ są one testowane manualnie.
- **Narzędzia:** xUnit

5.2. Przypadki testowe – tabela

Wypełnij tabelę przynajmniej dla 8 przypadków testowych:

ID testu	Testowany komponent	Dane wejściowe	Oczekiwany wynik	Actual wynik	Status (✓ / ✗)
TC-01	Wzorzec Stanu (Aircraft)	Samolot w <i>TaxiState</i> , paliwo 100%, komenda <i>TakeOff()</i>	Aktualny stan obiektu to <i>TakeOffState</i>	<i>TakeOffState</i>	OK
TC-02	Wzorzec Stanu (Aircraft)	Samolot w <i>TakeOffState</i> , osiągnięcie wysokości 1600 ft	Aktualny stan obiektu to <i>ClimbState</i>	<i>ClimbState</i>	OK
TC-03	Wzorzec Stanu (Aircraft)	Samolot w <i>CruiseState</i> , spadek poziomu paliwa do 4%	Aktualny stan maszyny to <i>EmergencyState</i>	<i>EmergencyState</i>	OK

TC-04	StatisticsHandler	Lista 3 różnych zdarzeń <i>FlightEvent</i> (w tym 1 anomalia, 1 awaria)	Wynik agregacji LINQ: 1 Anomaly, 1 Failure	1 Anomaly, 1 Failure	OK
TC-05	StatisticsHandler	Pusta lista zdarzeń wejściowych <i>FlightEvent</i> (brak logów)	Raport zwraca same zera dla wszystkich statystyk	Same zera	OK
TC-06	AutopilotCalculator	Obliczenie pochylenia (Pitch) dla skrajnych różnic wysokości docelowej	Funkcja czysta ogranicza wynik do bezpiecznego limitu: Max +8.0, Min -8.0	+8.0 / -8.0	OK
TC-07	AutopilotCalculator	Obliczenie ciągu (Throttle) dla ekstremalnych prędkości docelowych	Funkcja czysta ogranicza wynik przepustnicy od 0.0 do 1.0	0.0 / 1.0	OK
TC-08	AnomalyEngine	Samolot w <i>CruiseState</i> , wywołanie zdarzenia <i>EngineFailureAnomaly</i>	Obiekt anomalii zgłasza właściwość <i>IsActive = true</i>	true	OK

5.3. Szczególne przypadki testowe dla konceptów paradygmatu

- **Test niemutowalności i funkcji czystych:** Weryfikacja tego konceptu odbywa się w pliku *AutopilotTests.cs* (przypadki TC-06 i TC-07). Testowane metody, takie jak *CalculateNewPitch* czy *CalculateNewThrottle*, są w pełni czystymi funkcjami. Test weryfikuje, czy dla podanych argumentów wejściowych (np. docelowej prędkości) algorytm zwraca poprawną, deterministyczną wartość liczbową dla poprawek lotu.
- **Test polimorfizmu:** Są one zaimplementowane w pliku *AircraftIntegrationTests.cs* (przypadki TC-01, TC-02, TC-03). Testy te sprawdzają prawidłowe działanie wzorca projektowego Stan (State Pattern). Weryfikowane jest, czy główny obiekt samolotu (*Aircraft*) poprawnie, polimorficznie deleguje zachowanie do aktualnie przypisanej podklasy stanu (np. *TakeOffState*).
- **Test potoku przetwarzania (Pipelines) i braku mutacji stanu:** Jest to sprawdzane w klasie *StatisticHandler* (przypadek TC-04). Test sprawdza działanie deklaratywnych potoków zapytań LINQ służących do analizy danych z lotu. Do testowanej metody przekazywana jest

gotowa kolekcja zdarzeń (FlightEvent). Weryfikowane jest, czy potok prawidłowo przeprowadza serię transformacji: filtrowanie (filter), rzutowanie typów i agregację (reduce). Co najważniejsze z punktu widzenia FP, test

6. Indywidualny wkład członków grupy

Jest to sekcja obowiązkowa. Każdy student musi jednoznacznie udokumentować swój wkład. Niewystarczające uzasadnienie wkładu indywidualnego powoduje obniżenie oceny dla całej grupy lub dla konkretnego studenta.

⚠ ZASADA OCENIANIA

Ocena indywidualna każdego studenta zależy od jakości i zakresu jego udokumentowanego wkładu. Stwierdzenie „wszyscy pracowaliśmy wspólnie nad wszystkim” **nie jest akceptowane !!!!**

6.1. Student 1: Tomasz Orman

Zakres zrealizowanych zadań:	Pełnienie roli lidera zespołu — koordynacja pracy całej grupy, pilnowanie ustalonych terminów oraz bieżąca kontrola, czy poszczególne osoby realizują to, co było zaplanowane na dany etap projektu. Przegląd zgłaszanych przez zespół pull requestów oraz podejmowanie decyzji o ich scaleniu albo odrzuceniu (code review) — w roli osoby zarządzającej integracją kodu od poszczególnych członków zespołu do głównej wersji projektu. Implementacja głównych, integrujących komponentów systemu: klasy domenowej Aircraft (fasada delegująca do bieżącego stanu lotu), głównej pętli symulacji FlightController, modelu danych FlightData i AircraftConfig. Implementacja komponentów odpowiedzialnych za spajanie poszczególnych podsystemów: CascadeHandler (rozprzestrzenianie się kaskad uszkodzeń pomiędzy modułami), AnomalyEngine (cykl życia anomalii) oraz InputHandler (przekładanie akcji gracza na komendy). Implementacja warstwy prezentacji spinającej całość: Program.cs jako punkt kompozycji aplikacji oraz ekrany StartupScreen, ConsoleDashboardView, CameraView, FlightReportView, BlackBoxReadoutView. Refaktoryzacja kodu pod kątem czytelności i spójności — ujednolicenie konwencji nazewniczych i porządkowanie struktury
------------------------------	--

	katalogów po połączeniu pracy poszczególnych członków zespołu, a także usuwanie nieużywanych już fragmentów kodu.
Kod Źródłowy (pliki / klasy / funkcje):	AeroSimulator/Controllers/AnomalyEngine.cs AeroSimulator/Controllers/FlightController.cs AeroSimulator/Controllers/InputHandler.cs AeroSimulator/Core/Aircraft/Aircraft.cs AeroSimulator/Core/Aircraft/AircraftConfig.cs AeroSimulator/Core/Aircraft/AircraftSensors.cs AeroSimulator/Core/Aircraft/AircraftSystem.cs AeroSimulator/Core/Aircraft/DamageModel.cs AeroSimulator/Core/Aircraft/Enums/AnomalyType.cs AeroSimulator/Core/Aircraft/Enums/Difficulty.cs AeroSimulator/Core/Aircraft/Enums/EmergencyType.cs AeroSimulator/Core/Aircraft/Enums/FireLocation.cs AeroSimulator/Core/Aircraft/Enums/FireState.cs AeroSimulator/Core/Aircraft/Enums/FlightPhase.cs AeroSimulator/Core/Aircraft/Enums/PlayerAction.cs AeroSimulator/Core/Aircraft/Enums/SensorState.cs AeroSimulator/Core/Aircraft/Enums/Severity.cs AeroSimulator/Core/Aircraft/Enums/SystemStatus.cs AeroSimulator/Core/Aircraft/Enums/SystemType.cs AeroSimulator/Core/Aircraft/FlightData.cs AeroSimulator/Core/Aircraft/FlightDataSnapshot.cs AeroSimulator/Core/Aircraft/Sensors/AirspeedSensor.cs AeroSimulator/Core/Aircraft/Sensors/AltitudeSensor.cs AeroSimulator/Core/Aircraft/Sensors/EngineSensor.cs AeroSimulator/Core/Aircraft/Sensors/EngineTempSensor.cs AeroSimulator/Core/Aircraft/Sensors/FuelSensor.cs AeroSimulator/Core/Aircraft/Sensors/HydraulicSensor.cs AeroSimulator/Core/Aircraft/Sensors/ISensor.cs AeroSimulator/Core/Aircraft/Sensors/Sensor.cs AeroSimulator/Core/Aircraft/Sensors/SensorSystem.cs AeroSimulator/Core/Aircraft/Systems/AutopilotSystem.cs AeroSimulator/Core/Aircraft/Systems/ElectricalSystem.cs AeroSimulator/Core/Aircraft/Systems/EngineSystem.cs AeroSimulator/Core/Aircraft/Systems/FuelSystem.cs AeroSimulator/Core/Aircraft/Systems/HydraulicSystem.cs AeroSimulator/Core/Aircraft/Systems/IAvionicSystem.cs AeroSimulator/Core/Aircraft/Systems/NavigationSystem.cs AeroSimulator/Core/Aircraft/Systems/WeatherSystem.cs

	AeroSimulator/Core/Aircraft/Systems/WingSystem.cs
	AeroSimulator/Core/Commands/ActivateSystemCommand.cs
	AeroSimulator/Core/Commands/CommandHistory.cs
	AeroSimulator/Core/Commands/EmergencyDeclareCommand.cs
	AeroSimulator/Core/Commands/GoAroundCommand.cs
	AeroSimulator/Core/Commands/IFlightCommand.cs
	AeroSimulator/Core/Commands/ResolveAnomalyCommand.cs
	AeroSimulator/Core/Commands/SetAltitudeCommand.cs
	AeroSimulator/Core/Commands/SetHeadingCommand.cs
	AeroSimulator/Core/Commands/SetPitchCommand.cs
	AeroSimulator/Core/Commands/SetThrottleCommand.cs
	AeroSimulator/Core/Commands/ToggleAutopilotCommand.cs
	AeroSimulator/Core/Common/Option.cs
	AeroSimulator/Core/Events/AnomalyTriggeredEvent.cs
	AeroSimulator/Core/Events/CascadeTriggeredEvent.cs
	AeroSimulator/Core/Events/CommandExecutedEvent.cs
	AeroSimulator/Core/Events/EngineExplosionEvent.cs
	AeroSimulator/Core/Events/EngineFireEvent.cs
	AeroSimulator/Core/Events/EventBus.cs
	AeroSimulator/Core/Events/FlightEvent.cs
	AeroSimulator/Core/Events/GameOverEvent.cs
	AeroSimulator/Core/Events/Handlers/AlertBufferHandler.cs
	AeroSimulator/Core/Events/Handlers/AlertSystemHandler.cs
	AeroSimulator/Core/Events/Handlers/BlackBoxHandler.cs
	AeroSimulator/Core/Events/Handlers/CascadeHandler.cs
	AeroSimulator/Core/Events/Handlers/FlightLoggerHandler.cs
	AeroSimulator/Core/Events/Handlers/StatisticsHandler.cs
	AeroSimulator/Core/Events/LandingCompletedEvent.cs
	AeroSimulator/Core/Events/MaydayEvent.cs
	AeroSimulator/Core/Events/PlayerInputEvent.cs
	AeroSimulator/Core/Events/StateChangedEvent.cs
	AeroSimulator/Core/Events/SystemFailureEvent.cs
	AeroSimulator/Core/Events/TelemetryTickEvent.cs
	AeroSimulator/Core/Events/WingFireEvent.cs
	AeroSimulator/Core/States/ClimbState.cs
	AeroSimulator/Core/States/CriticalState.cs
	AeroSimulator/Core/States/CruiseState.cs
	AeroSimulator/Core/States/DescentState.cs
	AeroSimulator/Core/States/EmergencyState.cs
	AeroSimulator/Core/States/GroundState.cs
	AeroSimulator/Core/States/HoldingState.cs

	AeroSimulator/Core/States/IAircraftState.cs AeroSimulator/Core/States/LandingState.cs AeroSimulator/Core/States/TakeOffState.cs AeroSimulator/Core/States/TaxiState.cs AeroSimulator/Core/Strategies/Anomalies/AbstractAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/BirdStrikeAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/DecompressionAnomaly.cs
Koncepty paradygmatu zaimplementowane przez studenta:	Wzorzec State w czystej, w pełni polimorficznej formie — Aircraft.TransitionTo() oraz klasa Aircraft nie zawierają żadnej instrukcji warunkowej zależnej od fazy lotu (cała logika przejść jest rozproszona w klasach State). Wzorzec Facade/agregat — Aircraft jako centralny obiekt domenowy, do którego delegują wszystkie podsystemy (silniki, paliwo, czujniki, autopilot), spinający je w jeden konsekwentny model. Wzorzec Observer / Publish–Subscribe jako mechanizm rozprzestrzeniania kaskad uszkodzeń (CascadeHandler), odseparowujący logikę fizyki uszkodzeń od logiki wykrywania anomalii. Porządkowanie kodu pod względem zasad OOP (podział na warstwy, jednoznaczna odpowiedzialność klas) w ramach refaktoryzacji i scalania pracy zespołu.
Zaangażowanie (% szacunkowy):	35%
Link do commitów Git (opcjonalnie):	https://github.com/tomek2005/AeroSimulator/commits?author=tomek2005

6.1.1. Refleksja indywidualna – Student 1

Odpowiedz na poniższe pytania (każde min. 3–5 zdań). Odpowiedzi muszą być własne i autentyczne.

***Pytanie:** Który zastosowany koncept paradygmatu był dla Ciebie najtrudniejszy do zrozumienia i implementacji? W jaki sposób go opanowałeś/-aś?*

Najtrudniejsze było dla mnie scalenie wszystkich podsystemów samolotu w jedną spójną całość — tak, aby pogoda, anomalie i interakcje użytkownika rzeczywiście współdziałały ze sobą w czasie rzeczywistym, a nie funkcjonowały jako osobne, niezależne mechanizmy. Sam podział odpowiedzialności zbliżony do MVC nie był dla mnie od razu prosty do ułożenia i jeszcze chwilę się z nim męczyłem. Sporo problemów dały mi też czujniki, które miały celowo zwracać przekłamane (zazumione) wartości — musiałem dobrze zrozumieć, jak to sensownie zamodelować. Poradziłem sobie z tym głównie przez samodzielne szukanie rozwiązań w internecie, w tym oglądanie materiałów na YouTube. Ułatwiałem sobie też pracę, zostawiając na początku „puste” widoki, które jeszcze niczego nie robiły — dzięki temu mogłem najpierw uruchomić i przetestować samą logikę anomalii, mimo że nie dało się jeszcze z nimi nic zrobić z perspektywy gracza, a interfejs dopinałem stopniowo, krok po kroku.

Pytanie: Co zrobiłbyś/-abyś inaczej gdybyś projektował/-a projekt od nowa? Jakie decyzje architektoniczne okazały się słuszne, a jakie błędne?

Gdybym robił ten projekt od nowa, na początku zredukowałbym liczbę czujników i anomalii do minimum, żeby najpierw dopracować samą logikę i działanie systemu, a dopiero potem stopniowo dokładać kolejne elementy. Zrobiliśmy to w odwrotnej kolejności, co trochę utrudniło pracę, bo niemal cały kod trzeba było później refaktoryzować. Myślę też, że poszedłbym w stronę lepszej oprawy graficznej — zrezygnowałbym z konsoli i użył jakiegoś prostego silnika do gier, żeby zrobić chociaż minimalną grafikę, co zmieniłoby UX o wiele na lepsze.

Pytanie: Jak zastosowanie wybranego paradygmatu wpłynęło na jakość Twojego kodu w porównaniu z kodem imperatywnym?

Zastosowanie OOP spowodowało, że mogłem zacząć od małych, samodzielnych partii kodu, bez konieczności od razu realizowania wszystkiego naraz — na przykład wszystkich czujników jednocześnie — bo wystarczyło dodać kolejny plik (klasę), a resztą zajmowała się już istniejąca pętla i mechanizm obsługujący to automatycznie. W kodzie imperatywnym musiałbym za każdym razem tworzyć kolejne warunki i ręcznie sprawdzać kolejne przypadki. Poprawiło to też czytelność — łatwiej szło robić refaktoryzację i łatwiej było mi się orientować w kodzie, a osoba, która pisałaby po mnie, mimo złożoności projektu, powinna dość szybko się w nim odnaleźć.

Pytanie: Jakie wyzwania napotkałeś/-aś przy pracy zespołowej? W jaki sposób je rozwiązano?

Wyzwaniem, jakie napotkałem, było zarządzanie ludźmi — nauczanie ich, jak pracować w ramach wspólnego repozytorium, oraz pilnowanie, żeby wszystko było zrobione poprawnie i na czas. Samo koordynowanie projektem było chyba najtrudniejsze w całym projekcie. Dość trudnym problemem było też rozwiązywanie konfliktów, gdy ktoś długo tworzył swój fragment na osobnym branchu — po dłuższym czasie, przy próbie scalenia, pojawiało się sporo konfliktów. Wyzwaniem była również spójność kodu, bo każdy z nas pisze go nieco inaczej, ale na szczęście IDE mają wbudowane narzędzia do czyszczenia kodu i ujednolicania zapisu, dzięki czemu udało się w projekcie uzyskać jednolitość.

6.2. Student 2: Michał Iwanejko

Zakres zrealizowanych zadań:	-Zaimplementowanie stanów rozgrywki. -Implementacja FP w liczeniu statystyk i tworzeniu logów z lotu. -Dodanie mechaniki dystansu do lotniska docelowego. -Rozdzielenie typów lądowań na lądowania awaryjne i prawidłowe. -Testy weryfikujące poprawność danych samolotu w czasie rozgrywki (FlightData.cs). -Testy weryfikujące działanie autopilota w tym zaimplementowanych w niego funkcji czystych. -Zaimplementowanie testów integracyjnych maszyny stanów (State), weryfikujących automatyczne transycje faz lotu oraz komunikację międzymodułową. -Implementacja FP w systemie Autopilota.
Kod Źródłowy (pliki / klasy / funkcje):	AeroSimulator.Tests/Integration/AircraftIntegrationTests.cs AeroSimulator.Tests/Core/Events/Handlers/StatisticsHandlerTests.cs AeroSimulator/Core/States (cały folder)

	AeroSimulator/Core/Aircraft/Systems/AutopilotSystem.cs AeroSimulator.Tests/Core/Aircraft/Systems/AutopilotTests.cs AeroSimulator/Core/Events/Handlers/StatisticsHandler.cs AeroSimulator/Core/Events/Handlers/BlackBoxHandler.cs AeroSimulator/Views/BlackBoxReadoutView.cs AeroSimulator/Core/Aircraft/FlightData.cs AeroSimulator.Tests/Core/Aircraft/FlightDataTests.cs
Koncepty paradygmatu zaimplementowane przez studenta:	- FP: Zastowanie czystych funkcji w module AutopilotSystem. - FP: Wdrożenie deklaratywnych potoków przetwarzania danych i leniwej ewaluacji (LINQ jako odpowiednik map/filter/reduce) w klasie StatisticsHandler do wyliczania statystyk lotu. - OOP: Implementacja wzorca projektowego Stan (State Pattern).
Zaangażowanie (% szacunkowy):	25%
Link do commitów Git (opcjonalnie):	https://github.com/tomek2005/AeroSimulator/commits?author=altMichal

6.2.1. Refleksja indywidualna – Student 2

Odpowiedz na poniższe pytania (każde min. 3–5 zdań). Odpowiedzi muszą być własne i autentyczne.

Pytanie: *Który zastosowany koncept paradygmatu był dla Ciebie najtrudniejszy do zrozumienia i implementacji? W jaki sposób go opanowałeś/-aś?*

Najtrudniejsze do zaimplementowania dla Mnie było to czym musiałem zająć się w dużej części, czyli paradygmat funkcyjny. Nie miałem intuicji (i doświadczenia!), która pozwalałaby mi zrozumieć koncepty takie jak potoki danych czy funkcje wyższego rzędu. Opanowałem go po prostu eksperymentując i programując kolejne godziny, popełniając błędy i się z nich ucząc.

Pytanie: *Co zrobiłbyś/-abyś inaczej gdybyś projektował/-a projekt od nowa? Jakie decyzje architektoniczne okazały się słuszne, a jakie błędne?*

Na pewno błędem okazało się brak implementacji potoków danych w statystykach od początku. Implementowanie tego rozwiązania później było przez to znacznie bardziej kosztowne czasowo, natomiast bez wątplenia opłacalne. Byłaby to rzecz, którą na pewno zmieniłbym zaczynając projekt od nowa. Za najlepszą decyzję, którą podjąłem uważam implementację stanów gry od samego początku. Bardzo to ułatwiło późniejsze dobudowywanie reszty funkcjonalności i samo manualne testowanie gry na początkowym etapie tworzenia.

Pytanie: *Jak zastosowanie wybranego paradygmatu wpłynęło na jakość Twojego kodu w porównaniu z kodem imperatywnym?*

FP: Zastosowanie paradygmatu funkcyjnego spowodowało ogromną redukcję objętości kodu analitycznego. Zamiast pisać wielokrotnie zagnieżdżone pętle for/foreach i instrukcje warunkowe if do ręcznego zliczania błędów, wykorzystałem potoki LINQ, które są deklaratywne, zwięzłe i całkowicie wolne od efektów ubocznych (nie psują oryginalnych list).

OOP: Maszyna stanów (State Pattern) zlikwidowała problem “spaghetti code”. W kodzie imperatywnym musiałbym ciągle sprawdzać globalne flagi typu isFlying czy isEmergency, co jest wysoce podatne na błędy. Dzięki OOP każdy stan (np. wznoszenie, lot przelotowy) jest teraz izolowany we własnej klasie i hermetycznie zarządza własnymi regułami.

Pytanie: Jakie wyzwania napotkałeś/-aś przy pracy zespołowej? W jaki sposób je rozwiązano?

Wydaje mi się, że sama współpraca z zespołem była dla Mnie relatywnie nowym zjawiskiem, do którego musiałem się przystosować. Rozwiązywanie konfliktów na githubie czy podział pracy w zespole to ciężkie zadania ale ostatecznie udało nam się z nich wyjść na prostą, czego efekty widać w naszym projekcie. Myślę, że nie było jednego, uniwersalnego czynnika, który ułatwił nam pracę, a była to kwestia czasu spędzonego podczas wspólnych prac nad projektem.

6.3. Student 3: [Jakub Górski]

Zakres zrealizowanych zadań:	-Stworzenie asynchronicznej szyny zdarzeń (EventBus). -Zaprojektowanie systemu kaskadowych awarii bazującego na prawdopodobieństwie (CascadeHandler). -Utworzenie polimorficznych anomalii niszczących komponenty (np. WingFireAnomaly, ElectricalFailureAnomaly). -Wdrożenie struktury Option eliminującej błędy typu null. -Napisanie integracyjnych testów symulacyjnych sprawdzających pętlę czasu i awarie.
Kod Źródłowy (pliki / klasy / funkcje):	AeroSimulator/Core/Events/EventBus.cs AeroSimulator/Core/Events/Handlers/CascadeHandler.cs AeroSimulator/Core/Strategies/Anomalies/ AeroSimulator/Core/Common/Option.cs AeroSimulator.Tests/Core/Aircraft/Simulation/AnomalyAndEventTests.cs
Koncepty paradygmatu zaimplementowane przez studenta:	OOP: Wzorzec Obserwatora (EventBus), Wzorzec Strategii (IAnomaly), Polimorfizm. FP: Monada (Option), Niemutowalne typy danych (rekordy zdarzeń w C#), Domknięcia (lambdy w testach).
Zaangażowanie (% szacunkowy):	20%
Link do commitów Git (opcjonalnie):	[URL do profilu studenta w repozytorium lub lista hash commitów]

6.3.1. Refleksja indywidualna – Student 3

Odpowiedz na poniższe pytania (każde min. 3–5 zdań). Odpowiedzi muszą być własne i autentyczne.

Pytanie: Który zastosowany koncept paradygmatu był dla Ciebie najtrudniejszy do zrozumienia i implementacji? W jaki sposób go opanowałeś/-aś?

Najtrudniejszym konceptem do zrozumienia były monady z paradygmatu funkcyjnego (nasza klasa `Option<T>`). Ciężko było mi się przestawić z klasycznego, imperatywnego rzucania wyjątków na bezpieczniejsze funkcyjne owijanie wartości. Najbardziej w opanowaniu tego zagadnienia pomogły mi zajęcia laboratoryjne, na których to jedna lista dotyczyła dokładnie tego problemu. Przełożenie podstaw monad z laboratoriów na praktyczne zastosowanie w projekcie ostatecznie w pełni pozwoliło mi opanować ten koncept.

Pytanie: Co zrobiłbyś/-abyś inaczej gdybyś projektował/-a projekt od nowa? Jakie decyzje architektoniczne okazały się słuszne, a jakie błędne?

Gdybym projektował ten symulator od nowa, od samego początku w pełni odseparowałbym poszczególne czujniki samolotu, zamiast na wczesnym etapie silnie wiązać je z głównym stanem maszyny. Słusznym i

niezwykle wygodnym okazało się wdrożenie szyny zdarzeń (EventBus), która idealnie łączy się z generatorem kaskadowych uszkodzeń (CascadeHandler). Błędne okazało się mieszanie logiki obiektowej mutującej stan z funkcyjnymi potokami w jednej klasie, co doprowadziło do wielu frustrujących błędów. Gdybym od początku precyzyjnie rozdzielił odpowiedzialności, mógłbym tego uniknąć.

Pytanie: Jak zastosowanie wybranego paradygmatu wpłynęło na jakość Twojego kodu w porównaniu z kodem imperatywnym?

Zastosowanie hybrydy i wplecenie paradygmatu funkcyjnego do języka zorientowanego obiektowo drastycznie poprawiło czytelność oraz stabilność całego systemu awioniki. W czysto imperatywnym podejściu kaskadowe awarie szybko doprowadziłyby do powstania tzw. "spaghetti state", ponieważ różne systemy nadpisywałyby zmienne w niekontrolowany sposób. Dzięki wykorzystaniu w pełni niemutowalnych rekordów w zdarzeniach oraz domknięć w architekturze testowej, cała logika stała się przewidywalna i pozbawiona ukrytych efektów ubocznych.

Pytanie: Jakie wyzwania napotkałeś/-aś przy pracy zespołowej? W jaki sposób je rozwiązano?

Wyzwania jakie napotkałem w naszej pracy zespołowej to skomplikowane konflikty w repozytorium Git, wynikające z jednoczesnej modyfikacji ściśle powiązanych ze sobą plików bazowych. Bardzo często dochodziło do sytuacji, w której moje zaawansowane testy integracyjne rozjeżdżały się lokalnie, ponieważ opierały się na kodzie, który ktoś inny, w tym samym czasie modyfikował na innej gałęzi. Rozwiązaliśmy ten problem poprzez rygorystyczne wdrożenie procedury małych, zależnych od siebie Pull Requestów.

6.4. Student 4: [Przemysław Jaworski]

Zakres zrealizowanych zadań:	-Poprawianie kodu po błędach kolegów z grupy -Akceptowanie/odrzucając commitów -Pierwsze uruchomienie projektu bez interfejsu ale z działającymi systemami -Poprawianie projektu, aby spełniał założenia
Kod Źródłowy (pliki / klasy / funkcje):	- AeroSimulator/Core/Aircraft/Aircraft.cs - AeroSimulator/Core/Aircraft/AircraftConfig.cs - AeroSimulator/Core/Aircraft/AircraftSensors.cs - AeroSimulator/Core/Aircraft/AircraftSystem.cs - AeroSimulator/Core/Aircraft/DamageModel.cs - AeroSimulator/Core/Enums/Severity.cs - AeroSimulator/Core/FlightData.cs - AeroSimulator/Core/Systems/AutopilotSystem.cs - AeroSimulator/Core/Systems/ElectricalSystem.cs - AeroSimulator/Core/Systems/EngineSystem.cs - AeroSimulator/Core/Systems/FuelSystem.cs - AeroSimulator/Core/Systems/HydraulicSystem.cs - AeroSimulator/Core/Systems/IAvionicSystem.cs - AeroSimulator/Core/Systems/NavigationSystem.cs - AeroSimulator/Core/Systems/WeatherSystem.cs - AeroSimulator/Core/Systems/WingSystem.cs - AeroSimulator/Core/Events/EventBus.cs - AeroSimulator/Core/Events/FlightEvent.cs - AeroSimulator/Core/Events/GameOverEvent.cs - AeroSimulator/Core/Handlers/AlertSystemHandler.cs - AeroSimulator/Core/Handlers/BlackBoxHandler.cs - AeroSimulator/Core/Handlers/CascadeHandler.cs - AeroSimulator/Core/Handlers/FlightLoggerHandler.cs - AeroSimulator/Core/Handlers/StatisticsHandler.cs

	AeroSimulator/Core/Handlers/IFlightEventHandler.cs AeroSimulator/Core/Handlers/MaydayEvent.cs AeroSimulator/Core/States/GroundState.cs AeroSimulator/Core/Strategies/Anomalies/AbstractAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/BirdStrikeAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/DecompressionAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/ElectricalFailureAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/EngineFailureAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/EngineFireAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/FuelLeakAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/HydraulicFailureAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/IAomaly.cs AeroSimulator/Core/Strategies/Anomalies/IcingAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/MicroburstAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/RunwayIncursionAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/SensorFailureAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/TurbulenceAnomaly.cs AeroSimulator/Core/Strategies/Anomalies/WingFireAnomaly.cs AeroSimulator/Core/Weather/EnvironmentState.cs AeroSimulator/Core/Weather/WeatherCondition.cs AeroSimulator/Controllers/AnomalyEngine.cs AeroSimulator/Controllers/FlightController.cs AeroSimulator/Controllers/InputHandler.cs AeroSimulator/Infrastructure/AircraftFactory.cs AeroSimulator/Infrastructure/AnomalyFactory.cs AeroSimulator/Infrastructure/FlightLogger.cs AeroSimulator/Infrastructure/FlightReport.cs AeroSimulator/Infrastructure/SimulationConfig.cs AeroSimulator/Infrastructure/WeatherFactory.cs AeroSimulator/Program.cs
Koncepty paradygmatu zaimplementowane przez studenta:	OOP: Abstrakcja, Hermetyzacja (Enkapsulacja), Polimorfizm, Kompozycja, Dziedziczenie, Wzorce Projektowe (Design Patterns)
Zaangażowanie (% szacunkowy):	20%
Link do commitów Git (opcjonalnie):	https://github.com/tomek2005/AeroSimulator/commits/main/?author=PJ0506

6.4.1. Refleksja indywidualna – Student 4

Odpowiedz na poniższe pytania (każde min. 3–5 zdań). Odpowiedzi muszą być własne i autentyczne.

Pytanie: *Który zastosowany koncept paradygmatu był dla Ciebie najtrudniejszy do zrozumienia i implementacji? W jaki sposób go opanowałeś/-aś?*

Najtrudniejszym konceptem do pełnego zrozumienia i wdrożenia był dla mnie polimorfizm w połączeniu z Wzorcem Stanu (State Pattern), szczególnie w kontekście cyklu życia samolotu (IAircraftState). Było to skomplikowane i robiłem to po raz pierwszy sam bez wzorce, nie pomagała też sama ilość funkcji potrzebnych żeby samolot zaczął działać i złożoność tych systemów, z tego powodu polimorfizm okazał się dużym wyzwaniem.

Pytanie: *Co zrobiłbyś/-abyś inaczej gdybyś projektował/-a projekt od nowa? Jakie decyzje architektoniczne okazały się słuszne, a jakie błędne?*

Na pewno lepiej bym określił systemy jakie chcemy i końcowy wygląd aplikacji, dogadywanie tego w trakcie i różne wizje osób piszących ten sam kod, okazały się błędem. Prowadziło to do potrzeby częstej modyfikacji kodu i ogólnego zagmatwania systemu.

Pytanie: Jak zastosowanie wybranego paradygmatu wpłynęło na jakość Twojego kodu w porównaniu z kodem imperatywnym?

Zastosowanie paradygmatu obiektowego (OOP) zamiast klasycznego podejścia imperatywnego (proceduralnego) fundamentalnie zmieniło architekturę aplikacji, przekształcając potencjalny "spaghetti kod" w wysoce modułowy, bezpieczny i łatwy w utrzymaniu system. Pomogło to z dopisywaniem kodu, tworząc nowe pliki i rozszerzając bazowe założenia o np. lądowanie na lotnisku, podstawowe struktury były już gotowe zostało tylko uzupełnić nowy pomysł.

Pytanie: Jakie wyzwania napotkałeś/-aś przy pracy zespołowej? W jaki sposób je rozwiązano?

Potrzeba akceptowania Pull Requestów i otrzymywania tych akceptacji. Sprawdzanie kodu kolegów z grupy i rozwiązywanie konfliktów, naprawianie nie działającego kodu i przepisywanie go aby był zgodny z założeniami paradygmatu.

7. Wnioski i refleksja grupowa

7.1. Osiągnięte cele

Odnieś się do celów funkcjonalnych z sekcji 1.3. Które zostały w pełni zrealizowane, które częściowo, a które pominięto? Uzasadnij ewentualne odstępstwa.

Projekt zrealizował główne cele funkcjonalne: umożliwia wybór konfiguracji, prowadzi pętlę symulacji, obsługuje sterowanie, modeluje stany lotu, generuje anomalie, publikuje zdarzenia i tworzy raport końcowy. Szczególnie dobrze zrealizowano część dotyczącą architektury obiektowej, ponieważ każda istotna grupa odpowiedzialności ma osobny moduł. Zrealizowano także testy dla wielu kluczowych klas domenowych. Częściowo ograniczona pozostaje fizyczna dokładność symulacji, ponieważ model lotu jest uproszczony i nastawiony na grywalność oraz demonstrację architektury. Projekt nie jest profesjonalnym symulatorem aerodynamicznym, ale jako projekt z paradygmatów programowania spełnia cel pokazania złożonego, modularnego systemu.

7.2. Napotkane problemy i sposób ich rozwiązania

Opisz 3 największe trudności techniczne lub projektowe i jak je przezwyciężono:

- **Złożoność faz lotu i przejść między nimi:**

Opis problemu: samolot zachowuje się inaczej w zależności od fazy lotu. Na ziemi powinien mieć zerową prędkość pionową, przy starcie musi osiągnąć prędkości $V_1/V_R/V_2$, podczas wznoszenia rośnie wysokość, przy lądowaniu ważne są prędkości, pitch, roll i podwozie, a w sytuacji awaryjnej obowiązują inne reguły. Gdyby wszystkie te reguły były zapisane w jednej metodzie, kod stałby się trudny do czytania i testowania.

Rozwiązanie: zastosowano wzorec State. Utworzono interfejs `IAircraftState` i osobne klasy stanów, np. `GroundState`, `TaxiState`, `TakeOffState`, `ClimbState`, `CruiseState`, `LandingState`, `EmergencyState`. Klasa `Aircraft` deleguje zachowanie do aktualnego stanu. Dzięki temu logika przejść jest rozdzielona na mniejsze klasy i łatwiej wskazać, gdzie znajduje się zachowanie konkretnej fazy lotu.

- **Komunikacja między awariami, alertami, logami i black boxem:**

Opis problemu: pojedyncze zdarzenie w symulacji powinno mieć kilka skutków. Na przykład awaria silnika powinna pojawić się w alertach, zostać zapisana w black boxie, wpłynąć na statystyki, a czasem uruchomić kaskadę kolejnych awarii. Bez dobrego mechanizmu komunikacji klasy musiałyby bezpośrednio wywoływać siebie nawzajem, co prowadziłoby do silnego sprzężenia.

Rozwiązanie: zastosowano `EventBus` i wzorec `Observer`. Zdarzenia takie jak `AnomalyTriggeredEvent`, `SystemFailureEvent`, `EngineFireEvent`, `LandingCompletedEvent` i `GameOverEvent` są publikowane do centralnej szyny. `Handler` `BlackBoxHandler`, `CascadeHandler`, `AlertSystemHandler`, `FlightLoggerHandler` i inne reagują niezależnie. Dzięki temu dodanie nowego odbiorcy zdarzeń nie wymaga zmiany kodu publikującego zdarzenie.

- **Modelowanie awarii kaskadowych:**

Opis problemu: celem projektu było pokazanie, że awarie nie są izolowane. Bird strike może doprowadzić do pożaru silnika, pożar do uszkodzenia skrzydła, a uszkodzone skrzydło do asymetrycznego znoszenia i katastrofy. Trudność polegała na tym, żeby taki łańcuch nie był zapisany jako chaotyczna sekwencja instrukcji w jednym miejscu.

Rozwiązanie: awarie zostały opisane jako strategie `IAnomaly`, a przejścia kaskadowe są obsługiwane przez `CascadeHandler`. `Handler` nasłuchuje zdarzeń i na podstawie typu zdarzenia oraz prawdopodobieństwa publikuje kolejne zdarzenia albo modyfikuje model uszkodzeń. Dzięki temu logika kaskady jest skupiona w jednym module i nadal korzysta z architektury zdarzeniowej.

- **Bezpieczna obsługa uszkodzonych sensorów:**

Opis problemu: sensory mogą działać poprawnie, dawać zaszumione wartości, zawiesić się na ostatnim odczycie albo całkowicie przestać zwracać dane. Bez dobrego modelu odczytów łatwo byloby doprowadzić do błędów null albo do sytuacji, w której autopilot korzysta z nieistniejącej wartości.

Rozwiązanie: wprowadzono `Option<T>`, czyli opakowanie wartosci, ktore moze byc `Some` albo `None`. `Sensor.Read()` zwraca `Option<double>`, a autopilot wywołuje `IfPresent`, czyli aktualizuje sterowanie tylko wtedy, gdy odczyt faktycznie istnieje. To jest element podejścia funkcyjnego, który poprawia bezpieczeństwo logiki.

7.3. Ocena zastosowanego paradygmatu

Oceńcie retrospektywnie wybrany paradygmat. Odpowiedz na pytania:

- Czy paradygmat okazał się odpowiedni dla tego problemu? Co by się zmieniło, gdybyście użyli innego podejścia?

Tak, paradygmat obiektowy okazał się bardzo odpowiedni, ponieważ domena symulatora lotu naturalnie składa się z obiektów posiadających stan i zachowanie. Samolot, silniki, sensory,

autopilot, hydraulika, system paliwowy, pogoda, anomalie i stany lotu są łatwe do opisania jako klasy. OOP pozwoliło odwzorować strukturę problemu w strukturze kodu. Dzięki temu projekt jest bardziej zrozumiały: czytając katalogi i nazwy klas, widac, który fragment odpowiada za która część symulacji.

Gdyby projekt był napisany czysto proceduralnie, prawdopodobnie powstałaby jedna duża pętla symulacji z wieloma instrukcjami warunkowymi. Trudniej byłoby dodawać nowe anomalie, nowe stany lotu albo nowe typy pogody. Gdyby projekt był napisany czysto funkcyjnie, łatwiej byłoby testować transformacje danych, ale trudniej naturalnie reprezentować długotrwały, zmienny stan samolotu i systemów pokładowych. Dlatego najlepsze okazało się podejście hybrydowe: OOP do modelowania domeny, a elementy FP do bezpiecznych i testowalnych obliczeń.

- Jakimi były zalety wybranego paradygmatu (np. czytelność, modularność, testowalność)?
Największą zaletą była modularność. Można pracować nad stanami lotu, anomaliami, sensorami, widokami i komendami jako osobnymi częściami. Drugą zaletą to czytelność: EngineFireAnomaly od razu sugeruje, gdzie znajduje się logika pożaru silnika, a TakeOffState gdzie znajduje się logika startu. Trzecią zaletą to testowalność, bo wiele klas można uruchomić bez całego programu konsolowego. Czwartą zaletą to rozszerzalność: interfejsy IAnomaly, IWeatherStrategy, IAircraftState i IFlightCommand ułatwiają dodawanie nowych zachowań.
- Jakimi były ograniczenia lub trudności wynikające z paradygmatu?
OOP wprowadza więcej klas i plików, więc projekt wymaga dobrej organizacji folderów. Dla małego programu taka liczba wzorców mogłaby wydawać się nadmiarowa, ale w tym projekcie złożoność domeny ją uzasadnia. Pewnym ograniczeniem jest też to, że niektóre interfejsy są szerokie, np. IAircraftState, gdzie każdy stan implementuje wszystkie akcje, nawet jeśli część z nich jest pusta. Dodatkowo EventBus jako Singleton upraszcza komunikację, ale wprowadza globalny stan, który trzeba czyścić w testach.
- Czy hybryda OOP+FP wniosła wartość dodaną? Czy nie wprowadzała zbędnej złożoności?
Tak, hybryda wniosła wartość dodaną, ale FP jest tutaj dodatkiem, a nie głównym stylem architektonicznym. OOP odpowiada za model domenowy, natomiast elementy funkcyjne poprawiają bezpieczeństwo i testowalność wybranych fragmentów. Option<T> pozwala bezpiecznie modelować brak odczytu sensora. AutopilotCalculator ma metody, które działają jak funkcje czyste i są łatwe do testowania. LINQ pomaga filtrować anomalie i sensory bez ręcznego pisania długich pętli. Taka hybryda nie wprowadza zbędnej złożoności, ponieważ elementy funkcyjne są użyte tylko tam, gdzie realnie upraszczają kod.

7.4. Możliwe rozszerzenia projektu

Wskaż 3–5 kierunków, w których projekt mógłby być rozwinięty:

1. Dokładniejszy model aerodynamiczny z masą, siłą nośną, oporem, kątem natarcia i konfiguracją klapy.
2. Tryb zapisu i odtwarzania lotu na podstawie black box.
3. Graficzny interfejs użytkownika zamiast samej konsoli.
4. Edytor scenariuszy awaryjnych i tras lotu.

5. Rozbudowane testy end-to-end oraz raport pokrycia kodu.

8. Literatura i Źródła

Podaj co najmniej 5 Źródeł (podręczniki, artykuły naukowe, dokumentacja techniczna, oficjalna dokumentacja języka).

1. Microsoft Learn, C# documentation,
2. Microsoft Learn, .NET documentation,
3. xUnit documentation,
4. Gamma E., Helm R., Johnson R., Vlissides J., "Design Patterns: Elements of Reusable Object-Oriented Software"
5. Martin R. C., "Clean Architecture"
6. Kordziński, W., & Łakomy, M. (2020). Systemy sterowania lotem.

9. Karta oceny projektu (wypełnia prowadzący)

Poniższa tabela jest przeznaczona dla prowadzącego. Studenci nie wypełniają tej sekcji.

Kryterium	Opis	Max pkt	Uzyskane pkt
1. Wartość praktyczna i ciekawość zagadnienia	Problem jest nietrywialny, ma uzasadnienie praktyczne; projekt jest funkcjonalnie rozbudowany.	20	
2. Uzasadnienie wyboru paradygmatu	Uzasadnienie jest merytoryczne, szczegółowe i wynika z analizy domeny problemu (Sekcja 2).	20	
3. Jakość architektury i projektu systemu	Diagramy są poprawne (UML), architektura jest spójna, moduły mają dobrze zdefiniowane odpowiedzialności (Sekcja 3).	15	
4. Jakość implementacji i kodu	Kod jest czytelny, zgodny z zasadami paradygmatu; zastosowanie conceptów (HOF, closures, wzorce) jest poprawne i nietrywial. (Sekcja 4).	20	
5. Indywidualny wkład studentów	Każdy student ma wyraźny, udokumentowany i uzasadniony wkład (Sekcja 6).	20	
6. Jakość raportu	Raport jest kompletny, poprawnie napisany, spójny; zawiera wszystkie wymagane sekcje.	5	
SUMA		100	

Ocena końcowa: ____ / 100 pkt Stopień: ____ Data: ____ Podpis: ____

Załącznik A: Lista kontrolna przed złożeniem raportu

Przed oddaniem raportu każda grupa powinna przejść przez poniższą listę kontrolną. Odhaczcie każdą pozycję (✓) lub zaznaczcie jako nieukończoną (X).

Strona tytułowa i dane projektu

- ✓ Strona tytułowa wypełniona kompletnie (tytuł, nazwiska, języki, paradygmaty, data)

Sekcja 1 – Opis projektu

- ✓ Opis projektu (min. 5 zdań)
- ✓ Motywacja i kontekst (min. 3 zdania)
- ✓ Co najmniej 5 celów funkcjonalnych
- ✓ Wymagania niefunkcjonalne

Sekcja 2 – Uzasadnienie paradygmatu (KLUCZOWE)

- ✓ Uzasadnienie OOP zawiera konkretne przykłady klas/metod
- ✓ Uzasadnienie FP zawiera konkretne przykłady funkcji/potoków
- ✓ Uzasadnienie hybrydowe (jeśli dotyczy) jest szczegółowe
- ✓ Wymienione zasady SOLID z przykładami (jeśli OOP)
- ✓ Opisanie wzorce projektowe (jeśli zastosowane)

Sekcja 3 – Architektura

- ✓ Diagram architektury wysokiego poziomu
- ✓ Tabela komponentów wypełniona
- ✓ Diagram klas UML (jeśli OOP)
- ✓ Diagram potoku przetwarzania (jeśli FP)
- ✓ Drzewo katalogów projektu

Sekcja 4 – Implementacja

- ✓ Co najmniej 3 listingi kodu z omówieniem
- ✓ Każdy listing ma opis i analizę paradygmatyczną
- ✓ Tabela technologii wypełniona
- ✓ Opis obsługi błędów

Sekcja 5 – Testy (mile widziane)

- ✓ Co najmniej 8 przypadków testowych w tabeli
- ✓ Testy jednostkowe zaimplementowane i opisane
- ✓ Testy specyficzne dla paradygmatu opisane
- ✓ Podano procent pokrycia kodu

Sekcja 6 – Wkład indywidualny (KLUCZOWE)

- ✓ Każdy ze 3 – 4 studentów ma wypełnioną tabelę wkładu
- ✓ Każdy student odpowiedział na 4 pytania refleksyjne (min. 3 zdania każde)
- ✓ Lista plików / klas / funkcji przypisanych każdemu studentowi
- ✓ Szacunkowy % udziału każdego studenta podany

Sekcja 7 – Wnioski

- ✓ Odniesienie do celów funkcjonalnych z sekcji 1.3
- ✓ Opisano 3 problemy i ich rozwiązania
- ✓ Ocena paradygmatu retrospektywna
- ✓ Co najmniej 3 możliwe rozszerzenia

Sekcja 8 – Literatura

- ✓ Co najmniej 5 źródeł w formacie IEEE lub APA